Windows CMD Mastery

From Zero to Expert Windows Administration, Automation, Internals & Defensive Security Research

A Hands-On Command Prompt, Batch Scripting, Windows Administration and Security Research Field Guide

Volume 3

Parts XIX-XXIII: Defensive Security Research, Injection, Path Security, Incident Response, Forensics
Labs 71-80, 116-150

Copyright and Disclaimer

Copyright (c) Windows CMD Mastery. All rights reserved. This manuscript is educational material. Every command is provided for use on systems you own or are explicitly authorized to administer. Run every lab inside an isolated virtual machine with a snapshot taken first. Commands that change system configuration, security settings, disks, boot data or encryption state are marked DANGER and can cause data loss. The author and publisher accept no liability for damage resulting from use of this material. Windows behavior differs between versions, editions and builds; verify each command against current Microsoft Learn documentation for your exact build. No malware, credential theft, evasion or unauthorized-access content is included; security chapters use benign simulations only.

How to Use This Book

Read in order the first time. Type every example; do not paste. Every lab lists its own objective, prerequisites, expected result and cleanup. Work inside a disposable VM.

Long lines in listings. Code is wrapped to fit the page. A wrapped line continues on the next line, which starts with `..` and extra indentation. Join it back into **one** line when you type it; the `..` marker is not part of the command.

Legend	Meaning
[B] Beginner	No prior knowledge assumed
[I] Intermediate	Needs the earlier chapters
[A] Advanced	Needs parser and batch fundamentals
[X] Expert	Internals, edge cases, design trade-offs

Callout	Meaning
SAFETY (green)	How to keep a lab contained and reversible
DANGER / CAUTION (red)	Can destroy data, change boot or security state, or needs a snapshot
SECURITY (orange)	Defensive implication, abuse potential, detection note
NOTE (blue)	Clarification or version note

Table of Contents

Copyright and Disclaimer 2

How to Use This Book 2

Table of Contents 3

Part XIX - Defensive Security Research 6

Chapter 44. Security Research Method and Telemetry 8

44.1 Ground rules . . . 8
44.2 The analysis template used for every technique . . . 8
44.3 Telemetry: what you can actually see . . . 8
44.4 Reading a process-creation event . . . 9
44.4a Signal strength . . . 9

Chapter 45. Technique Catalog (Defensive View) 11

45.1 Technique 1: command chaining and complex command lines . . . 11
45.2 Technique 2: masquerading (name and path) . . . 11
45.3 Technique 3: search order and executable resolution . . . 11
45.4 Technique 4: file-extension ambiguity . . . 12
45.5 Technique 5: environment manipulation . . . 12
45.6 Technique 6: unsafe batch variables and command injection . . . 13
45.7 Technique 7: quoting vulnerabilities (unquoted paths and arguments) . . . 13
45.8 Technique 8: service abuse concepts . . . 13
45.9 Technique 9: scheduled-task abuse concepts . . . 14
45.10 Technique 10: registry autostart and configuration concepts . . . 14
45.11 Technique 11: suspicious process chains and parent/child relationships . . . 14
45.12 Technique 12: trust evaluation: signed versus unsigned, and hash verification . . . 15
45.13 Technique 13: privilege-boundary mistakes . . . 15
45.14 Technique 14: secret exposure in command lines, environment and logs . . . 16

Chapter 46. Dual-Use Utilities ("Living off the Land") - Defensive Cards 17

46.1 Why these tools get special treatment . . . 17
46.2 Reading a LOLBin alert: a worked example . . . 24

Part XX - Command Injection and Batch Security 26

Chapter 47. Command Injection in Batch Files 28

47.1 The core problem . . . 28
47.2 Example 1: a "harmless" ping tool . . . 28
47.3 The fix: validate with an allowlist, then use data as data . . . 29
47.4 Example 2: file names, wildcards and path traversal . . . 29
47.5 Example 3: call and percent injection . . . 30
47.6 Example 4: second-order injection through generated scripts . . . 31
47.7 Example 5: trusting the environment . . . 31
47.8 Secure-pattern summary . . . 32
47.9 Reviewing a batch script for injection (checklist) . . . 32

Part XXI - Path and Command Resolution Security 36

Chapter 48. Resolution Order and Hardening 38

48.1 The resolution pipeline, security view . . . 38
48.2 Verification habits . . . 38
48.3 Auditing the PATH for writable folders . . . 38
48.4 Hardening checklist . . . 39
48.5 A related class: DLL search order (concept) . . . 39
48.6 A safe demonstration of unquoted-path probing with CMD itself . . . 39

Security Labs 41

Chapter 49. Defensive Security Labs 116-135 43

Part XXII - Incident Response 57

Chapter 50. Incident Response Method 59

50.1 What incident response is (and is not) . . . 59
50.2 Ground rules ("do no harm") . . . 59
50.3 Evidence handling . . . 59
50.4 Questions and where the answers live . . . 60
50.5 Scoping and timeline building . . . 60
50.6 Containment, eradication, recovery (order of operations) . . . 60
50.7 Verifying cleanup . . . 61

50.8 Reporting template 61

Part XXIII - Windows Forensics (Command-Line Collection) 62

Chapter 51. Collecting and Handling Evidence 64

51.1 Volatile versus persistent evidence 64

51.2 Trusted tools and a small footprint 65

51.3 A volatile-evidence collector 65

51.4 Reading the evidence: what each artifact tells you 67

51.5 Validating a collection 68

Chapter 52. Simulated Incident and IR Labs 69

52.1 The simulation (benign) 69

Batch Scripting Labs (continued) 78

Chapter 53. Batch Labs 71-80 80

Back Matter (Volume 3) 87

Chapter 54. Glossary Additions 89

Chapter 55. Command Index (Volume 3) 90

Chapter 56. References and Verification Notes 91

Chapter 57. Lab Inventory After Volume 3 92

Chapter 58. What Comes Next 93

Part XIX - Defensive Security Research

How Windows command-line functionality is misused, how it shows up in telemetry, and how to detect, mitigate and investigate it, using only benign simulations in isolated labs.

Chapter 44. Security Research Method and Telemetry

44.1 Ground rules

> **SAFETY.** Everything in this volume is for **authorized, isolated lab use**: a virtual machine you own, with a snapshot, with no network path to anything you do not control. Lab artifacts are harmless (they print text, write a log line, or ping loopback). This volume contains **no malware, no credential theft, no evasion or bypass techniques and no persistence payloads**. Where an attacker technique is described, the book shows the *underlying Windows behavior* and then spends the rest of the page on detection, mitigation and investigation.

If a lab asks you to create something that looks like a persistence mechanism (a scheduled task or a service entry), it points to a harmless script, uses a `CmdLab` name, is never started as a service, and has a cleanup step.

44.2 The analysis template used for every technique

```
Normal functionality        what the feature is for
        |
Security risk               what goes wrong if it is misused or misconfigured
        |
Why it can be abused        the design property that makes it possible
        |
Observable behavior         what appears in processes, files, registry, network, logs
        |
Safe lab reproduction       a harmless simulation of the *behavior*
        |
Detection                   telemetry, rules, baselines
        |
Mitigation                  configuration, policy, architecture
        |
Investigation               how to scope and confirm
```

44.3 Telemetry: what you can actually see

Defense depends on **what is logged**. Most useful sources on a Windows host:

Source	What it records	Enabled by default?
Security log 4688 (process creation)	New process, creator, token elevation; **command line only if the policy is on**	Off (needs audit policy)
Sysmon (Sysinternals) event 1	Process create with full command line, parent image and command line, hashes, integrity level, user	Not installed by default
Sysmon events 3, 11, 12/13, 22	Network connection, file create, registry change, DNS query	Per configuration
System 7045 / Security 4697	Service installed	7045 on
Security 4698-4702	Scheduled task created/deleted/enabled/disabled/updated	Off (Object Access audit)
TaskScheduler/Operational	Task registered, started, completed	**Off** (enable the log)
PowerShell/Operational 4104	Script blocks executed	Off (policy)

Source	What it records	Enabled by default?
WMI-Activity/Operational	WMI provider activity	On (limited)
Windows Firewall log, 5156/5157	Allowed/blocked connections	Off
Defender/Operational	Detections, setting changes	On
Security 1102, System 104	Log cleared	On

Enabling command-line capture for process-creation events (a configuration change; reversible; do it in the lab first):

```
auditpol /get /subcategory:"Process Creation"
auditpol /set /subcategory:"Process Creation" /success:enable
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
    .. ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
```

Rollback: `auditpol /set /subcategory:"Process Creation" /success:disable` (if it was off before) and `reg delete "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v ProcessCreationIncludeCmdLine_Enabled /f`. In production, set this through Group Policy: *Advanced Audit Policy > Detailed Tracking > Audit Process Creation* and *Administrative Templates > System > Audit Process Creation > Include command line in process creation events*.

> **SECURITY. Command lines are sensitive.** Capturing them is essential for detection, but they will also capture passwords and tokens that people (wrongly) pass as arguments. Restrict who can read the Security log and the log collector, and fix the scripts that leak (Chapter 45, technique 14).

44.4 Reading a process-creation event

A 4688 event (Windows 10/11 with command-line capture) contains fields such as: **SubjectUserName** (who started it), **NewProcessName** (full image path), **CommandLine**, **ParentProcessName** (creator process; present on current builds), **TokenElevationType** (default, full or limited), **NewProcessId**. Sysmon event 1 adds hashes and the parent's command line.

Query recent ones:

```
wevtutil qe Security "/q:*[System[(EventID=4688)]]" /c:5 /rd:true /f:text
```

Questions to ask of every command line: **Who ran it? From what parent? From which path? With what arguments? Was it elevated? Does the combination make sense for this user and time?** One field is rarely decisive; the combination is.

44.4a Signal strength

Signal	Typical meaning	Strength alone
System binary running from the wrong folder	Masquerading or a copy	High
Interpreter (`cmd`, `wscript`, `mshta`, `rundll32`) launched by Office, browser or PDF reader	Document or download executing code	Medium-high
Long, odd, repetitive command line	Obfuscation or generated code	Medium

Signal	Typical meaning	Strength alone
New service or task shortly before other anomalies	Persistence candidate	Medium
Outbound network connection from a normally offline utility	Staging or tool transfer	Medium
Unsigned binary	Many legitimate tools are unsigned	Low
`cmd.exe` running at all	Extremely common	None

Combine signals; never alert on one weak signal without context.

Chapter 45. Technique Catalog (Defensive View)

Each technique below follows the template. "Safe lab" points to a lab in Chapter 49 or an earlier lab.

45.1 Technique 1: command chaining and complex command lines

Aspect	Detail
Normal	`cmd /c a & b && c` runs several commands from one process; installers and scripts rely on it
Risk	Complex, nested or heavily escaped lines hide intent from casual reading and simple string rules
Why abusable	CMD has no schema for commands; any text can be a command line (Chapter 10)
Observable	A `cmd.exe` (or script host) with a long line using many `&`, `^`, `%var:~n,m%` substitutions; child processes spawned in quick succession
Safe lab	Lab 125 (rules against synthetic lines)
Detection	Log the **expanded** command line (4688/Sysmon); alert on structure (very long, nested `cmd /c`, high caret or substring-expansion counts) plus context (parent, user, path), not on a single word
Mitigation	Restrict interpreters to those who need them (AppLocker/WDAC), constrained admin tools, code-signing requirements for scripts
Investigation	Reconstruct the line by expanding it in a safe environment **as text** (replace the action with `echo`) so nothing executes; trace child processes

45.2 Technique 2: masquerading (name and path)

Aspect	Detail
Normal	Windows processes have well-known names and live in `System32`
Risk	A binary named like a system process (`svchost.exe`, `lsass.exe`) in another folder, or a name with a look-alike spelling, blends into process lists
Why abusable	Process names are labels; users and tools often check only the name
Observable	Image path outside `%SystemRoot%\System32`; wrong parent; no signature; hash does not match the known file
Safe lab	Lab 117
Detection	Rule: well-known system image names outside their expected folders; parent/child baseline (Section 26.3); signer and hash checks
Mitigation	Application control; block execution from user-writable folders; file integrity monitoring
Investigation	Full path, hash, signer, parent, creation time of file, who wrote it

45.3 Technique 3: search order and executable resolution

Aspect	Detail
Normal	Typing a name searches the current directory, then PATH, trying PATHEXT extensions (Chapter 12)

Aspect	Detail
Risk	A same-named file in an earlier searched folder runs instead of the intended tool
Why abusable	Search order is a convenience, not an authenticity check
Observable	Process image path differs from the expected one; `where tool` lists multiple results
Safe lab	Labs 47, 119, 121, 122
Detection	Alert on system tool names executing from non-system paths; audit PATH folders for write access by non-admins
Mitigation	Absolute paths in scripts; `NoDefaultCurrentDirectoryInExePath`; no user-writable folders in system PATH; app control
Investigation	`where`, process path, which folder was current, who owns/wrote the shadowing file (Chapter 48 expands)

45.4 Technique 4: file-extension ambiguity

Aspect	Detail
Normal	Explorer hides known extensions by default; scripts and executables share double-click behavior
Risk	`invoice.pdf.cmd` displays as `invoice.pdf`; icons can mislead; many extensions run code (`.exe .com .bat .cmd .vbs .js .wsf .hta .ps1 .scr .lnk .msi .jar` and others)
Why abusable	Display convention versus actual extension; file associations decide what runs
Observable	Files with double extensions; scripts in download/temp folders; `.lnk` files with command lines
Safe lab	Lab 118
Detection	Regex rule for names ending in two extensions where the last is executable; alerts on script extensions arriving via email/web; Mark-of-the-Web (`Zone.Identifier`) present
Mitigation	Show file extensions via policy; open `.js .vbs .hta` in an editor by default; block risky attachments; SmartScreen/Mark-of-the-Web handling
Investigation	`dir /r`, `assoc`/`ftype`, hash, sender/origin from Zone.Identifier

45.5 Technique 5: environment manipulation

Aspect	Detail
Normal	Variables such as `PATH`, `COMSPEC`, `TEMP` configure programs; children inherit them
Risk	A changed `PATH` or `COMSPEC` (or a script variable such as an install directory) redirects which program a trusting script launches
Why abusable	Scripts trust inherited environment; no integrity check
Observable	Unexpected `PATH` entries; `COMSPEC` not `C:\Windows\System32\cmd.exe`; persistent values in `HKCU\Environment` or `HKLM\...\Session Manager\Environment`
Safe lab	Labs 120, 121
Detection	Baseline machine and user environment values; alert on change (`reg query` diff); alert on `setx` of PATH/COMSPEC
Mitigation	Set a clean `PATH` and use absolute paths at the top of privileged scripts; protect the registry keys with ACLs; avoid running privileged scripts in user context

Aspect	Detail
Investigation	`reg query HKCU\Environment`, machine environment key, process environment (Process Explorer), who changed it (Sysmon 12/13, 4657)

45.6 Technique 6: unsafe batch variables and command injection

substitute user or file input into commands	
ining `&	< > ^ " %` changes what executes (Chapter 47)
ands **before** parsing, so data becomes syntax	
sses spawned from a script run; unexpected files created by a "simple" tool	
elemetry), 123-124, Chapter 47 labs	
patterns (`set /p`, `%1` inside commands, `call` with data); monitor children of the service/tool	
th allowlists; use delayed expansion for data; avoid shelling out with concatenated input; prefer with real argument arrays	
with an `echo`-only harness; examine logs for the input value	

45.7 Technique 7: quoting vulnerabilities (unquoted paths and arguments)

Aspect	Detail
Normal	Paths with spaces must be quoted; Windows tries to interpret unquoted paths in stages
Risk	An unquoted service path `C:\Program Files\My App\svc.exe` makes Windows probe `C:\Program.exe`, then `C:\Program Files\My.exe`, then the real file; a writable earlier location can intercept
Why abusable	Ambiguous parsing combined with a writable parent folder
Observable	Service `ImagePath` unquoted with spaces; folder ACLs allowing user writes
Safe lab	Labs 88 and 122
Detection	Audit `ImagePath`/scheduled task actions for unquoted, space-containing paths; ACL check on each probed location
Mitigation	Quote paths; install under protected folders (`Program Files`); tighten ACLs; WDAC
Investigation	`sc qc`, `icacls` of parent folders, file creation times

45.8 Technique 8: service abuse concepts

Aspect	Detail
Normal	Services start early, run as high-privilege accounts, restart on failure
Risk	A new or modified service gives persistent, privileged execution; a service whose binary or configuration is user-writable allows hijack

Aspect	Detail
Why abusable	Privileged context plus automatic start plus configuration stored in the registry
Observable	Event 7045/4697; new `Services\<name>` key; unusual `ImagePath`, `ObjectName`, `Start`; service binary in user-writable paths
Safe lab	Labs 87, 88, 130
Detection	Alert on every new service; compare against baseline; check signer/path; check `sc sdshow` for weak ACLs
Mitigation	Least-privilege service accounts, protected install folders, restrict who may create services, application control
Investigation	Creation event, creating process and user, binary hash/signature, dependency/start history (7036/7040)

45.9 Technique 9: scheduled-task abuse concepts

Aspect	Detail
Normal	Maintenance, updates, backups
Risk	Tasks can run as SYSTEM, at boot or logon, even hidden; useful for persistence and delayed execution
Why abusable	Creating a task needs only the right to the task folder; actions are arbitrary commands
Observable	4698 / 106 / 140 events; new XML under `System32\Tasks`; TaskCache registry entries
Safe lab	Labs 99, 100, 129
Detection	Alert on creation by unexpected users, tasks outside vendor folders running scripts from user-writable paths, SYSTEM principals, `ONLOGON`/`ONSTART` triggers on workstations
Mitigation	Restrict task creation; app control for the task action; monitor task store integrity
Investigation	Section 31.2 procedure: preserve XML, verify target, correlate events, contain, remove, verify

45.10 Technique 10: registry autostart and configuration concepts

Aspect	Detail
Normal	Run keys, Services, Winlogon and file-association keys configure startup and handlers
Risk	Writable or modified entries cause attacker-chosen code to run at logon or at handler invocation
Why abusable	HKCU needs no admin; many keys are consulted at every logon
Observable	New values in autostart locations (Section 30.7); Sysmon 12/13; 4657 where auditing is configured
Safe lab	Lab 131
Detection	Baseline-and-diff of autostart keys; alert on new entries pointing to user-writable paths
Mitigation	Application control; least privilege; monitoring; Autoruns reviews
Investigation	`reg export` the key, verify each referenced file, hash/signature, creation time, author process

45.11 Technique 11: suspicious process chains and parent/child relationships

Aspect	Detail
Normal	Section 26.3 lists expected parents
Risk	Document, browser or archive tools launching shells; shells launching network tools
Why abusable	The parent link reveals how code arrived, which is why it is useful and why attackers try to break or fake it
Observable	Sysmon event 1, 4688 creator process; process tree in EDR
Safe lab	Lab 126
Detection	Rules over parent/child pairs plus command-line context; baselines per role
Mitigation	ASR rules and application control that block Office child processes; email/web attachment controls
Investigation	Walk the tree up and down; verify the parent is the same process (creation times); find the originating file

45.12 Technique 12: trust evaluation: signed versus unsigned, and hash verification

Aspect	Detail
Normal	Code signing proves who published code and that it has not changed since signing; hashes identify exact file content
Risk	Treating "signed" as "safe" or "unsigned" as "bad" both fail; attackers use signed system tools; legitimate internal tools are often unsigned
Why abusable	Trust decisions based on one attribute are easy to satisfy
Observable	`Get-AuthenticodeSignature` status; signer name; certificate validity; hash mismatch against a known-good list
Safe lab	Labs 127, 128
Detection	Maintain hash baselines for critical paths; alert on new executables in protected paths; check signer allowlists
Mitigation	WDAC/AppLocker with publisher and hash rules; catalog files; signing internal scripts
Investigation	Hash lookups against internal inventory and reputable threat-intelligence sources; compare with vendor-published hashes

45.13 Technique 13: privilege-boundary mistakes

Aspect	Detail
Normal	Privileged code (services, SYSTEM tasks, admin scripts) should read only inputs that unprivileged users cannot influence
Risk	A privileged script that reads a config file, temp file, DLL or PATH entry in a **user-writable** location, or writes predictably named files into one, lets a less-privileged user steer it
Why abusable	Convenience: shared folders, `%TEMP%`, `C:\Users\Public`, `C:\ProgramData` with default ACLs
Observable	SYSTEM process opening or executing files under user-writable paths; ACLs granting Users/Authenticated Users write on folders used by privileged code
Safe lab	Labs 121, 130

Aspect	Detail
Detection	ACL audits on every folder referenced by a service, task or privileged script; Process Monitor to see where a privileged process looks (NAME NOT FOUND in writable places)
Mitigation	Move privileged inputs to admin-only folders; use `%SystemRoot%\Temp`-style protected locations or per-run protected directories; validate inputs; least privilege
Investigation	Identify the privileged process, the file it consumed, who could write it, and whether it was modified

45.14 Technique 14: secret exposure in command lines, environment and logs

Aspect	Detail
Normal	Scripts need credentials and tokens
Risk	Command lines are visible to other users on the machine (`Get-CimInstance Win32_Process`, Task Manager) and are captured by 4688/Sysmon and by monitoring agents; environment variables are inherited by every child; logs and transcripts record whatever was printed
Why abusable	Command lines and environments are not confidential by design
Observable	Arguments like `/password:...`, `-p secret`, `token=` in process listings and event logs; `echo %PASSWORD%` in log files; `set` output in tickets
Safe lab	Labs 123, 124
Detection	Search command-line telemetry and log files for secret-looking patterns; scan scripts for credentials
Mitigation	Prompt for secrets (and clear them); use the Windows Credential Manager/DPAPI, managed identities or vaults; pass secrets via protected files or stdin; scrub logs; rotate any secret that appeared in a log
Investigation	Treat any exposed secret as compromised; rotate and review where it was used

NOTE. The 14 techniques above cover what MITRE ATT&CK catalogs as, among others, T1059.003 (Windows Command Shell), T1036 (Masquerading), T1574 (Hijack Execution Flow, including path interception), T1053.005 (Scheduled Task), T1543.003 (Windows Service), T1547.001 (Registry Run Keys / Startup Folder), T1070.001 (Clear Event Logs), T1552 (Unsecured Credentials). Use the ATT&CK site for the full list of procedures and detections.

Chapter 46. Dual-Use Utilities ("Living off the Land") - Defensive Cards

46.1 Why these tools get special treatment

Windows ships hundreds of signed Microsoft tools. A subset can **download, decode, load or launch other code**. They are legitimate and often necessary, and for the same reason they appear in intrusions: they are already present, signed, trusted and unlikely to be blocked. Security people call this "living off the land" and use the term **LOLBin** (living-off-the-land binary). The public **LOLBAS** project catalogs them, and MITRE ATT&CK maps their misuse (for example T1218 System Binary Proxy Execution, T1105 Ingress Tool Transfer, T1197 BITS Jobs, T1047 WMI).

Defensive principle for every card below: **the tool is not the threat; the unusual use of the tool is.** Detection works on *context*: who ran it, from which parent, with which arguments, touching which paths and network destinations, at what time. Each card therefore lists: legitimate use, why it is dual-use, risk, detection and mitigation. These cards deliberately do **not** include working abuse command lines.

SECURITY. Block-by-name is brittle (copies can be renamed) and breaking (you may disable something you need). Prefer **application control** (WDAC or AppLocker with publisher/hash/path rules), **attack surface reduction (ASR) rules**, least privilege, egress filtering and good telemetry. Test any block in audit mode first.

CERTUTIL [External, certificates and encoding]

Field	Detail
Purpose / Why	Manages certificates, certificate stores, CAs and revocation; also hashes and encodes files
Syntax	`certutil -hashfile file [MD5|SHA1|SHA256|SHA512]` ; `certutil -encode in out` ; `certutil -decode in out` ; `certutil -verify cert` ; `certutil -store My` ; `certutil -dump file`
Beginner	`certutil -hashfile "%WINDIR%\System32\notepad.exe" SHA256`
Intermediate	`certutil -store My` (list your personal certificates)
Advanced	`certutil -dump cert.cer` (parse and print a certificate or any binary as hex)
Real-world	Verifying a download's hash; inspecting a certificate chain during a TLS problem
Output	`SHA256 hash of file:` then hex digits, `CertUtil: -hashfile command completed successfully.`
Exit / Errors	0 success; nonzero HRESULT such as `0x80070002` (file not found)
Admin	No for hashing and user stores; yes for machine stores and CA operations
Versions	Present on all supported Windows versions
Mistakes	Hashing a file while it is still being written; comparing hashes case-sensitively or with spaces
Dual-use nature	The same binary can also encode and decode arbitrary files as text and retrieve content from URLs through its cached-URL features
Risk	Ingress of tools or staging of data using a trusted signed binary

Field	Detail
Detection	`certutil.exe` command lines containing URL-fetch or encode/decode switches; certutil making outbound network connections; certutil writing executables or scripts to user-writable paths; Sysmon 1/3
Mitigation	Egress filtering by process; application control; Defender ASR/behavior detections; allow certutil only for administrators who need it
Safe lab / Cleanup	Labs 85, 127 (hashing); no cleanup
Related	`Get-FileHash`, `sigcheck`
PowerShell	`Get-FileHash -Algorithm SHA256`, `Get-ChildItem Cert:\CurrentUser\My`

BITSADMIN [External, Background Intelligent Transfer Service]

Field	Detail
Purpose / Why	Creates and monitors BITS transfer jobs: resumable background downloads and uploads that survive reboots and network changes
Syntax	`bitsadmin /list [/allusers] [/verbose]` (read-only); job creation and modification options exist but are **deprecated**
Beginner	`bitsadmin /list /allusers`
Intermediate	`bitsadmin /list /allusers /verbose` (shows job owner, files, notify command)
Advanced	Compare with `powershell -NoProfile -Command "Get-BitsTransfer -AllUsers"`
Real-world	Inspecting why an update-style transfer keeps retrying
Output	Job GUID, display name, state (QUEUED, CONNECTING, TRANSFERRING, SUSPENDED, ERROR, TRANSFERRED, CANCELLED), owner, file counts
Exit / Errors	Access denied for other users' jobs without elevation
Admin	Elevation needed to list all users' jobs
Versions	**Deprecated** in favor of the PowerShell BITS cmdlets; still present on Windows 10/11
Mistakes	Assuming an empty list proves no BITS activity (jobs are per-user and may be complete)
Dual-use nature	Jobs are persistent, run in the background and can launch a program when a transfer finishes or fails (notification command)
Risk	Persistent transfer or a stealthy trigger to run a program
Detection	`bitsadmin` processes with job-creation or notify-command options; BITS jobs with unexpected URLs or a notify command; **Microsoft-Windows-Bits-Client/Operational** events (job created, transfer started and stopped; the log must be reviewed or forwarded); Sysmon network events from `svchost` BITS
Mitigation	Prefer PowerShell cmdlets for legitimate use; restrict BITS via Group Policy where feasible; egress control; periodic review of `Get-BitsTransfer -AllUsers`
Safe lab / Cleanup	Lab 133 reads jobs only
Related	`Get-BitsTransfer`, `curl.exe`
PowerShell	`Get-BitsTransfer`, `Start-BitsTransfer`, `Remove-BitsTransfer`

REGSVR32 [External, COM registration]

Field	Detail
Purpose / Why	Registers or unregisters COM server DLLs (writes class information to the registry)
Syntax	`regsvr32 [/s] [/u] [/i[:cmdline]] [/n] dllname`
Beginner	`regsvr32 /?` (help dialog; changes nothing)
Intermediate	`regsvr32 /s /u vendor.dll` (silently unregister a vendor component during uninstall)
Advanced	Reading what a DLL registered: `reg query HKCR\CLSID /s /f vendor.dll`
Real-world	Installer and repair routines registering a component
Output	A message box on success or failure (suppressed by `/s`)
Exit / Errors	0 success; 3 DLL load failure; 4 entry point not found
Admin	Yes for machine-wide registration
Versions	32-bit copy in `SysWOW64`, 64-bit in `System32`
Mistakes	Registering the wrong bitness; unregistering a system component
Dual-use nature	It **loads and executes code from a DLL** (and can host certain script components) on behalf of the caller
Risk	Signed-binary proxy execution; network-sourced or user-path DLLs
Detection	`regsvr32.exe` with `/s` and `/i` plus unusual arguments; arguments that look like URLs; DLLs from user-writable paths or `Temp`; regsvr32 making network connections; Office/script parents; writes to `HKCU\Software\Classes\CLSID`
Mitigation	Application control for DLLs and regsvr32 itself on endpoints where not needed; ASR rules; egress filtering; monitor COM registration changes
Safe lab / Cleanup	Lab 125 (synthetic-log detection); do not run it against unknown DLLs
Related	`rundll32`, `reg query HKCR`
PowerShell	None (use installer logic); `Get-ChildItem HKLM:\SOFTWARE\Classes\CLSID` for inventory

RUNDLL32 [External, DLL function launcher]

Field	Detail
Purpose / Why	Calls an exported function in a DLL; used by Control Panel items, shell extensions and installers
Syntax	`rundll32 dllname,EntryPoint [arguments]`
Beginner	`rundll32 shell32.dll,Control_RunDLL desk.cpl` (opens Display settings on older layouts)
Intermediate	`rundll32 printui.dll,PrintUIEntry /?` (print management help)
Advanced	Inventory legitimate use by auditing shortcuts and scheduled tasks that call it
Real-world	Opening classic Control Panel pages from a script

Field	Detail
Output	The function's UI or nothing
Exit / Errors	Often silent; an error box if the DLL or entry point is missing
Admin	Depends on the function
Versions	32/64-bit copies
Mistakes	Calling unknown exports (can crash or change settings)
Dual-use nature	Runs arbitrary exported code from any DLL path
Risk	Execution of malicious DLLs under a trusted process name
Detection	`rundll32.exe` with **no arguments**, with DLLs in `Temp`, `AppData`, `Downloads`, `ProgramData`, with ordinal references (`#1`) to unknown DLLs, or making network connections; parent processes such as Office or script hosts; unusual child processes
Mitigation	Application control with DLL rules; ASR rules; EDR behavior detection; restrict write access to folders from which DLLs load
Safe lab / Cleanup	Lab 125
Related	`regsvr32`, `control.exe`
PowerShell	`Start-Process control.exe -ArgumentList ...` for Control Panel items

MSHTA [External, HTML Application host]

Field	Detail
Purpose / Why	Runs HTML Applications (`.hta`): HTML plus script with full local trust, outside a browser's sandbox
Syntax	`mshta file.hta`
Beginner	`mshta /?` (shows nothing useful; the tool is rarely needed)
Intermediate	Check who still uses it: `reg query HKCR\.hta` and `assoc .hta`
Advanced	Block-readiness test: inventory `.hta` files under `%ProgramFiles%` and user profiles
Real-world	A few legacy line-of-business tools ship HTAs
Output	A window (or none)
Exit / Errors	Script errors displayed in a dialog
Admin	No
Versions	Present on Windows 10/11; Microsoft is retiring some legacy script technologies over time, so check current Windows documentation
Mistakes	Opening an HTA from email "to view it"
Dual-use nature	An HTA runs script with the user's rights and can start any program
Risk	Document-to-code delivery via a signed host
Detection	`mshta.exe` with URL arguments, with a script-protocol pseudo-URL, from Office, browser or script-host parents; `mshta` spawning `cmd`/PowerShell/`wscript`; network connections

Field	Detail
Mitigation	Remove or block `mshta.exe` on endpoints without a business need (application control); change the `.hta` default handler to Notepad via policy; ASR rules
Safe lab / Cleanup	Labs 118, 125
Related	`wscript`, `cscript`, `assoc`
PowerShell	None

WMIC [External, WMI command line (deprecated)]

Field	Detail
Purpose / Why	Command-line interface to Windows Management Instrumentation (WMI): query hardware, OS, processes, users, and invoke WMI methods locally or remotely
Syntax	`wmic <alias> get <properties>` ; `wmic /node:host ...`
Beginner	`wmic os get Caption,Version`
Intermediate	`wmic process get ProcessId,ParentProcessId,Name,CommandLine`
Advanced	`wmic logicaldisk get Name,Size,FreeSpace`
Real-world	Legacy scripts that gather inventory
Output	Aligned columns
Exit / Errors	`'wmic' is not recognized` on builds where it is removed or disabled
Admin	Remote and some queries need rights
Versions	**Deprecated**; disabled by default on Windows 11 24H2 and later (verify on your build)
Mistakes	Building new scripts on it; using it where PowerShell CIM cmdlets are available
Dual-use nature	WMI can **create processes** (locally or remotely) and run queries on schedule; the data it exposes is valuable to defenders and to attackers
Risk	Remote execution and reconnaissance using built-in management plumbing (ATT&CK T1047)
Detection	`wmic.exe` with process-creation or `/node:` options; **WmiPrvSE.exe** spawning unexpected children; Microsoft-Windows-WMI-Activity/Operational events (such as 5857-5861); permanent WMI event subscriptions (inventory with `Get-CimInstance -Namespace root\subscription -ClassName __EventConsumer`)
Mitigation	Remove the WMIC feature where unused; restrict remote WMI/DCOM by firewall and group membership; monitor subscriptions; use PowerShell CIM cmdlets for legitimate work
Safe lab / Cleanup	Read-only queries only; see Lab 133 for the subscription inventory
Related	`Get-CimInstance`, `systeminfo`, `tasklist`
PowerShell	`Get-CimInstance Win32_OperatingSystem`, `Get-CimInstance Win32_Process`

FORFILES [External, file selection and command execution]

Field	Detail
Purpose / Why	Selects files by name, date or type and runs a command for each
Syntax	`forfiles [/p path] [/m mask] [/s] [/d [+|-]date|days] [/c "command"]`
Beginner	`forfiles /p "%WINDIR%\Logs" /m *.log` (just lists)
Intermediate	`forfiles /p "%TEMP%" /d -30 /c "cmd /c echo @path"` (preview of files older than 30 days)
Advanced	Replace `echo` with `del` only after reviewing the preview
Real-world	Age-based cleanup of log and temp folders
Output	One line per file when the command prints
Exit / Errors	`ERROR: No files found with the specified search criteria.` (exit 1)
Admin	Only for protected paths
Versions	Same
Mistakes	Running a destructive command without the preview step; quoting problems in `/c`
Dual-use nature	The `/c` option runs an arbitrary command, so it can act as an indirect launcher
Risk	Command execution through a benign-looking file iterator
Detection	`forfiles.exe` whose `/c` argument starts shells, interpreters or network tools rather than file maintenance; unusual parents
Mitigation	Restrict on endpoints that do not need it; review scheduled tasks that use it
Safe lab / Cleanup	Section 22.5, Lab 32
Related	`for /r`, `robocopy /minage`
PowerShell	`Get-ChildItem | Where-Object LastWriteTime -lt (Get-Date).AddDays(-30)`

MSIEXEC [External, Windows Installer]

Field	Detail
Purpose / Why	Installs, repairs and removes Windows Installer packages (`.msi`, `.msp`)
Syntax	`msiexec /i package.msi [/qn] [/l*v log.txt]`; `msiexec /x package.msi`; `msiexec /?`
Beginner	`msiexec /?`
Intermediate	`msiexec /i vendor.msi /qn /l*v "%TEMP%\vendor_install.log"` (silent install with a verbose log)
Advanced	Read `%TEMP%\vendor_install.log` for return value 3 lines to find the failing action
Real-world	Silent software deployment

Field	Detail
Output	Nothing in `/qn` mode; a log file; Application log events from `MsiInstaller`
Exit / Errors	0 success; 3010 success, reboot required; 1603 fatal error; 1618 another install is running; 1605 product not installed
Admin	Yes for per-machine installs
Versions	Same
Mistakes	`/qn` without a log leaves no diagnostics; installing from untrusted shares
Dual-use nature	Installs run with elevated rights and execute custom actions; packages can be fetched from remote locations
Risk	Execution of attacker-supplied packages in a trusted signed host; privilege escalation if the policy `AlwaysInstallElevated` is enabled
Detection	`msiexec` with remote package locations or user-writable paths, run silently, from unexpected parents; MsiInstaller events (11707 success, 11708 failed); Defender and SmartScreen reputation
Mitigation	**Ensure** `AlwaysInstallElevated` **is not enabled** (Lab 133); restrict installs to signed packages and software-distribution tools; application control
Safe lab / Cleanup	Lab 133 (policy check, read-only)
Related	`DISM`, `wusa`, `Get-Package`
PowerShell	`Start-Process msiexec -ArgumentList ... -Wait`, `Get-Package`

INSTALLUTIL [External (.NET), assembly installer]

Field	Detail
Purpose / Why	Runs the installer classes inside a .NET assembly (for example, registering a .NET Windows service)
Syntax	`InstallUtil.exe [/u] assembly.exe` (located under `%WINDIR%\Microsoft.NET\Framework64\v4.0.30319\`, not on PATH)
Beginner	`dir "%WINDIR%\Microsoft.NET\Framework64\v4.0.30319\InstallUtil.exe"` (confirm presence)
Intermediate	Developers use it to install .NET service executables
Advanced	Check which tasks, services or installers reference it
Real-world	Legacy .NET Windows service deployments
Output	Installer log lines
Exit / Errors	Nonzero on installer exceptions
Admin	Usually yes
Versions	Part of .NET Framework 4.x on Windows 10/11
Mistakes	Running it on assemblies you did not build
Dual-use nature	It loads an assembly and **runs code during install and uninstall**
Risk	Proxy execution of managed code through a signed .NET host

Field	Detail
Detection	`InstallUtil.exe` outside normal deployment contexts, with assemblies in user-writable folders, from script or Office parents, with options that suppress console output
Mitigation	Application control; developer-tool restrictions on workstations; monitoring
Safe lab / Cleanup	Read-only presence check; nothing to clean up
Related	`sc create`, `New-Service`
PowerShell	`New-Service` for service creation

CSCRIPT and WSCRIPT [External, Windows Script Host]

Field	Detail
Purpose / Why	Windows Script Host (WSH) engines for `.vbs`, `.js`, `.wsf`: `cscript` writes to the console, `wscript` shows dialog boxes
Syntax	`cscript //nologo script.vbs` ; `//B` batch mode (no prompts), `//E:engine`, `//T:seconds` timeout, `//H:cscript` set default host
Beginner	`cscript //nologo hello.vbs` where `hello.vbs` contains `WScript.Echo "hello"`
Intermediate	`cscript //nologo inventory.vbs > inv.txt`
Advanced	Check whether WSH is enabled: `reg query "HKLM\SOFTWARE\Microsoft\Windows Script Host\Settings" /v Enabled` (absent means enabled)
Real-world	Legacy admin and logon scripts
Output	Script output on the console (`cscript`) or in dialogs (`wscript`)
Exit / Errors	Script engine error messages with line numbers; exit code from `WScript.Quit(n)`
Admin	No
Versions	VBScript is being phased out as a Windows feature (it has become a Feature on Demand in newer Windows 11 builds); verify on your build
Mistakes	Double-clicking scripts received by email; leaving `.js` and `.vbs` associated with the script host for ordinary users
Dual-use nature	Scripts run with the user's rights and have broad access to files, registry, network and COM objects
Risk	The most common delivery vehicle for script-based malware via email and downloads
Detection	`wscript.exe` or `cscript.exe` launched from `Downloads`, `Temp`, archives, mail clients, or Office; scripts that make network connections or spawn shells; double-extension files
Mitigation	Disable WSH where unused (set `Enabled` to 0 in the Settings key under HKLM and HKCU); change default handlers for `.js .vbs .vbe .wsf .hta` to Notepad by policy; application control; block script attachments
Safe lab / Cleanup	Lab 125 and a benign `hello.vbs` demonstration in Lab 118
Related	`mshta`, `powershell`, `assoc`
PowerShell	PowerShell itself (constrained language mode, script signing, logging)

46.2 Reading a LOLBin alert: a worked example

Synthetic alert: "`rundll32.exe` *started by* `winword.exe`, *command line contains a DLL path under* `C:\Users\alice\AppData\Local\Temp`, *followed by an outbound connection from* `rundll32.exe`."

Question	Why it matters	How to answer
Is the parent plausible?	Word does not normally launch `rundll32`	Process tree (Sysmon 1 / 4688), user was in a document
Is the DLL trusted?	A DLL in Temp is unusual	Hash, signature, creation time, origin (`dir /r` for Zone.Identifier)
What did it do next?	Scope of compromise	Network (Sysmon 3, firewall log), children, file and registry writes
What else is on the host?	Persistence and lateral movement	Section 31.2 (tasks), 27.2 (services), 30.7 (registry), Chapter 50
What is the response?	Containment before cleanup	Isolate the host, preserve evidence, then remediate

Notice that the decision uses **combinations of facts**, never the name `rundll32` alone.

Part XX - Command Injection and Batch Security

Safe, deliberately vulnerable examples that use a harmless marker command, followed by the corrected versions. The goal is secure programming and detection.

Chapter 47. Command Injection in Batch Files

47.1 The core problem

Command injection happens when input from a less-trusted source is placed into a command line and the shell treats part of it as *syntax* instead of *data*. In batch files the cause is Phase 1 of the parser (Chapter 10): `%VAR%` and `%1` are replaced **before** CMD decides which characters are operators. A value containing `&`, `|`, `<`, `>`, `^`, `"` or `%` can therefore change what runs.

> **SAFETY.** Every injection in this chapter uses the marker `echo INJECTED-MARKER`. If you see that line, the injection worked; nothing else happens. Never replace the marker with a real command, and do not test against systems you do not own.

Who is the "attacker"? Anyone who can influence an input of a script that runs with **more privilege than that person has**: a web form feeding a script, a file name dropped in a shared folder, a CSV from another team, a scheduled task reading a user-writable file, a helpdesk tool passing a ticket field. If the person who supplies the input could simply type the command themselves with the same rights, nothing is gained; **the risk is a privilege or trust boundary**.

47.2 Example 1: a "harmless" ping tool

Save as `pingtool_bad.cmd`:

```
@echo off
set /p HOST=Host to test:
ping -n 1 %HOST%
```

Run it and enter `127.0.0.1`. It pings. Run it again and enter:

```
127.0.0.1 & echo INJECTED-MARKER
```

Output: the ping reply **and** `INJECTED-MARKER`. Explanation: `set /p` stores the text raw. Then `%HOST%` is pasted into the line, which becomes `ping -n 1 127.0.0.1 & echo INJECTED-MARKER`; `&` separates two commands.

Other inputs and effects (all benign to try):

Input	Becomes	Lesson
`127.0.0.1 && echo INJECTED-MARKER`	conditional second command	`&&`, `||` work too
`127.0.0.1 > out.txt`	redirects output to a file	file write primitive
`-t`	`ping -n 1 -t`	**argument injection**: user-controlled switches change tool behavior
`%COMPUTERNAME%`	expanded only if passed through `call` or typed at a prompt	percent injection, Section 47.5

47.3 The fix: validate with an allowlist, then use data as data

Save as `pingtool_good.cmd`:

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
set "HOST="
set /p "HOST=Host to test: "
if not defined HOST (echo No input & exit /b 2)
if not "!HOST:~63!"=="" (echo Too long & exit /b 2)

rem --- validate: write the value to a file WITHOUT letting CMD parse it,
rem --- then test the file with an anchored allowlist pattern
set "VF=%TEMP%\pt_%RANDOM%%RANDOM%.tmp"
>"%VF%" echo(!HOST!
findstr /r /x "[A-Za-z0-9][A-Za-z0-9.-]*" "%VF%" >nul
set "OK=%ERRORLEVEL%"
del "%VF%" 2>nul
if not "%OK%"=="0" (echo Rejected: not a valid host name & exit /b 2)

"%SystemRoot%\System32\PING.EXE" -n 1 !HOST!
exit /b %ERRORLEVEL%
```

Walkthrough:

- `set /p "HOST=..."` keeps the value in a variable without parsing it.
- `!HOST!` (delayed expansion) inserts the value **after** parsing, so `&` in the text stays inert while it is written to the validation file.
- `findstr /r /x` with a character-class pattern is an **allowlist**: the whole line must be letters, digits, dots and hyphens. The first character must be alphanumeric, so a value like `-t` is rejected (no argument injection).
- Only after validation is `!HOST!` placed in the command line. At that point the value cannot contain special characters.
- The tool is called by **absolute path**, not by name (Chapter 48).
- A length limit bounds the input.

Tests to run (expected result in brackets): `127.0.0.1` [pings], `localhost` [pings], `127.0.0.1 & echo INJECTED-MARKER` [rejected], `-t` [rejected], `a"b` [rejected], empty [No input], 80 `a` characters [Too long].

> **NOTE. Why not just add quotes?** `ping -n 1 "%HOST%"` fixes `&` but fails when the value itself contains a quote (`x" & echo INJECTED-MARKER & "`), because quotes cannot be escaped inside quote mode (Section 13.2). Quoting reduces risk; validation removes it.

47.4 Example 2: file names, wildcards and path traversal

A tool meant to show files from one public folder:

```
@echo off
type "%TEMP%\cmdlab\public\%~1"
```

Setup (harmless data):

```
md "%TEMP%\cmdlab\public" "%TEMP%\cmdlab\secret"
echo public data> "%TEMP%\cmdlab\public\a.txt"
echo FAKE-SECRET-FOR-LAB> "%TEMP%\cmdlab\secret\s.txt"
```

Call	What happens
`show_bad a.txt`	prints `public data` (intended)
`show_bad ..\secret\s.txt`	**path traversal**: prints the fake secret from outside the public folder
`show_bad *.txt`	**wildcard expansion**: prints every matching file

Fixed version: accept only a plain file name, then confirm the resolved path stays inside the base folder.

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
set "BASE=%TEMP%\cmdlab\public"
set "NAME=%~1"
if not defined NAME (echo Usage: %~nx0 filename & exit /b 2)

set "VF=%TEMP%\sf_%RANDOM%%RANDOM%.tmp"
>"%VF%" echo(!NAME!
findstr /r /x "[A-Za-z0-9._-][A-Za-z0-9._-]*" "%VF%" >nul
set "OK=%ERRORLEVEL%"
del "%VF%" 2>nul
if not "%OK%"=="0" (echo Rejected file name & exit /b 2)
if "!NAME:..=!" neq "!NAME!" (echo Rejected: dots sequence & exit /b 2)

if not exist "%BASE%\!NAME!" (echo Not found & exit /b 1)
type "%BASE%\!NAME!"
```

Notes: the allowlist excludes `\ / : * ? " < > |` and space, which kills traversal, drive switching, alternate-stream syntax (`a.txt:stream`), wildcards and redirection. The `..` check blocks names made only of dots and sequences like `a..b` (extra strictness). `if exist` before `type` gives a clean error. A caller passing the argument unquoted can still break the *calling* line (`show_good a.txt & echo X`), so the caller must quote or validate too: **validation belongs at every trust boundary, not just inside the script**.

47.5 Example 3: `call` and percent injection

```
@echo off
set /p MSG=Message:
call :log "%MSG%"
exit /b 0

:log
echo [LOG] %~1
exit /b 0
```

Enter `%COMPUTERNAME%` (type it literally; at the `set /p` prompt it is not expanded). Output: `[LOG]` followed by **your computer's name**. The first pass pasted `%MSG%` (containing `%COMPUTERNAME%`); `call` then performed a **second percent expansion**, resolving the variable. Any environment variable's value can be disclosed this way, and `call` also doubles carets and re-parses `&` in some situations.

Fix: do not pass untrusted data as `call` arguments. Keep it in a variable and read it with delayed expansion inside the subroutine:

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
set "MSG="
set /p "MSG=Message: "
call :log
exit /b 0

:log
echo [LOG] !MSG!
exit /b 0
```

Entering `%COMPUTERNAME%` now prints exactly `[LOG] %COMPUTERNAME%`.

> **DANGER / CAUTION.** Delayed expansion has its own hazard: **values containing** `!` are consumed or misparsed when delayed expansion is on during assignment. Assign untrusted values with delayed expansion **off**, then enable it only where you read them (`setlocal DisableDelayedExpansion` ... `setlocal EnableDelayedExpansion`).

47.6 Example 4: second-order injection through generated scripts

A tool builds a script from a list of names, then runs it:

```
@echo off
setlocal DisableDelayedExpansion
> make.cmd echo @echo off
for /f "skip=1 tokens=1 delims=," %%n in (users.csv) do >> make.cmd echo mkdir
    .. "%%TEMP%%\cmdlab\home\%%n"
call make.cmd
```

With a normal `users.csv` it creates folders. If one line in the CSV is:

```
alice" & echo INJECTED-MARKER & "
```

the generated line becomes `mkdir "...\alice" & echo INJECTED-MARKER & ""`, and running `make.cmd` executes the marker. The FOR variable is substituted **after** parsing, so the generating line was safe, but the **text written to disk is parsed again later**. Lessons: **generated code is a second trust boundary**; validate the data before it is written (allowlist), or better, do not generate scripts: perform the action directly (`for /f ... do mkdir "..."` using a validated value).

47.7 Example 5: trusting the environment

A privileged script calls `tool.exe` by bare name and builds paths from `%TEMP%` or `%APPDATA%`. Whoever controls the environment, the current directory or a PATH folder controls what runs (Chapter 48). A **clean-room header** for scripts that run with elevated rights:

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
set "PATH=%SystemRoot%\System32;%SystemRoot%"
set "PATHEXT=.COM;.EXE;.BAT;.CMD"
set "NoDefaultCurrentDirectoryInExePath=1"
cd /d "%~dp0"
rem call system tools by absolute path where it matters:
rem "%SystemRoot%\System32\whoami.exe" /groups
```

This removes inherited PATH surprises, fixes the extension order and stops the current directory from being searched first. Also: do not read configuration or code from user-writable locations when running with higher privileges; do not write predictable file names in shared folders.

47.8 Secure-pattern summary

Hazard	Insecure pattern	Secure pattern
Raw input into commands	`ping %HOST%`	Validate with an anchored allowlist, then use
Quotes as the only defense	`"%VALUE%"`	Allowlist; quotes cannot be escaped in CMD
Argument injection	value starts with `-` or `/`	Require alphanumeric first character; use `--`-style separators where the tool supports them
Paths from input	`type "%BASE%\%1"`	Allowlist file names; no `\ / : * ?`; verify existence; compare resolved path to base
`call` with data	`call :f "%DATA%"`	Keep data in a variable; delayed expansion inside the subroutine
`!` in data	delayed expansion on at assignment	Assign with it off, enable only to read
Generated scripts	`echo cmd %%x >> run.cmd` then run	Validate before writing, or avoid generation
Inherited environment	bare tool names	Clean-room header; absolute paths
Temp files	`%TEMP%\out.txt` in a shared place	Per-run unique name in a protected folder
Secrets	on the command line	Prompt, stdin, protected file, credential store
Failure handling	continue on error	Check `%ERRORLEVEL%`; exit with a code

47.9 Reviewing a batch script for injection (checklist)

1. Find every **input source**: `%1`..`%9`, `%*`, `set /p`, `for /f` over files or command output, environment variables, files read from shared locations.
2. Follow each value to every **sink**: command lines, `call`, `cmd /c`, file paths, generated scripts, registry writes.
3. Is the value **validated by allowlist** before reaching a sink? Is validation done on the raw value or after a parse that already damaged it?
4. Is the script run with **more privilege** than the input provider? If not, the finding is low risk.
5. Does it call tools by **name** from an uncontrolled PATH or current directory?

Quick searches (at the prompt) to locate candidates:

```
findstr /n /i /l /c:"set /p" /c:"%*" /c:"cmd /c" /c:"call " /c:"for /f" *.cmd *.bat
findstr /n /i /l /c:"%1" /c:"%2" *.cmd *.bat
```

Detection from the outside: process telemetry showing a **service or scheduled script spawning unexpected children**, or child processes whose command lines contain fragments of user input.

Lab 146. Reproduce a command injection with a harmless marker

Objective. See data become syntax. **Difficulty.** [A]. **Prerequisites.** Chapters 10 and 47. **Environment. VM**, scratch folder, standard prompt.

Commands. Save Example 1 as `pingtool_bad.cmd`. Run it five times, entering in turn: `127.0.0.1`; `127.0.0.1 & echo INJECTED-MARKER`; `127.0.0.1 && echo INJECTED-MARKER`; `127.0.0.1 > inj.txt`; `-t` (press Ctrl+C after a few replies).

Expected result. Normal ping; ping plus marker; ping plus marker; no on-screen output and a file `inj.txt` containing the ping output; a continuous or repeated ping (conflicting switches can behave differently by build; record what you see). **Explanation.** Each variant shows a different effect of unvalidated expansion: command chaining, redirection and argument injection. **Verification.** `type inj.txt`. **Troubleshooting.** If nothing is injected, confirm you typed the input at the `Host to test:` prompt, not on the command line. **Cleanup.** `del pingtool_bad.cmd inj.txt`. **Challenge.** Predict the behavior for `127.0.0.1 | more` before testing. **Solution.** The pipe feeds ping's output to `more`; it displays normally and waits, showing that even `|` changes execution.

Lab 147. Fix it with an allowlist and prove it with tests

Objective. Write and test the corrected tool. **Difficulty.** [A]. **Prerequisites.** Lab 146. **Environment. VM**, scratch folder.

Commands. Save Section 47.3's `pingtool_good.cmd`. Create `tests.txt` with one test input per line (the seven inputs listed in the section, writing the long one as 80 letters). Run each by piping the line to the script (`echo input| pingtool_good` for the simple ones; for inputs containing `&` use the interactive prompt).

Expected result. Valid names ping; invalid ones print `Rejected` or `Too long` and return code 2, and **no marker appears**. **Explanation.** Allowlist validation happens before the value reaches the command line. **Verification.** `echo %ERRORLEVEL%` after each run (0 or the ping code for valid input, 2 for rejections). **Troubleshooting.** Piping `&` through `echo` itself parses the `&`; use the interactive prompt for those cases. **Cleanup.** `del pingtool_good.cmd tests.txt`. **Challenge.** Extend the allowlist to accept bracketed IPv6 literals safely. **Solution.** Decide the exact grammar first (for example hex digits, colons and brackets in an anchored pattern), then add a separate rule; do not loosen the existing character class.

Lab 148. Traversal and wildcard injection

Objective. Reproduce and fix path traversal and wildcard expansion. **Difficulty.** [A]. **Prerequisites.** Section 47.4. **Environment. VM**, scratch folder.

Commands. Run the setup from Section 47.4; save the vulnerable and fixed scripts as `show_bad.cmd` and `show_good.cmd`; run `show_bad a.txt`, `show_bad ..\secret\s.txt`, `show_bad *.txt`, then the same three against `show_good`, then `show_good a.txt:stream` and `show_good ""`.

Expected result. The vulnerable tool leaks `FAKE-SECRET-FOR-LAB` and lists all files; the fixed tool prints only `public data` for `a.txt` and rejects every other form. **Explanation.** The allowlist removes every character that

gives a name path meaning. **Verification.** The secret text never appears from `show_good`. **Troubleshooting.** If `show_good ..\secret\s.txt` appears to work, check that the allowlist line is intact. **Cleanup.** `rd /s /q "%TEMP%\cmdlab\public" "%TEMP%\cmdlab\secret" & del show_bad.cmd show_good.cmd`. **Challenge.** Add support for subfolders safely. **Solution.** Accept a restricted relative path grammar, resolve it with `for %%f in ("%BASE%\!NAME!") do set "FULL=%%~ff"`, and verify that `FULL` starts with `%BASE%\`.

Lab 149. `call` and percent injection

Objective. Show second-pass expansion and fix it. **Difficulty.** [A]. **Prerequisites.** Section 47.5. **Environment. VM**, scratch folder.

Commands. Save the vulnerable and the fixed logger as `log_bad.cmd` and `log_good.cmd`. Run each and enter `%COMPUTERNAME%`, then `%USERNAME%`, then `a^&b`.

Expected result. The vulnerable script prints the computer name and user name instead of the typed text and may mangle the third input; the fixed script prints each input exactly as typed. **Explanation.** `call` expands percent signs a second time, so data that looks like a variable is resolved. **Verification.** Compare outputs side by side. **Troubleshooting.** The `^` is literal input at a `set /p` prompt. **Cleanup.** `del log_bad.cmd log_good.cmd`. **Challenge.** Explain what an attacker learns from the vulnerable version. **Solution.** They can read the value of any environment variable by naming it (paths, user names, sometimes secrets placed in variables), which discloses information and aids further attacks.

Lab 150. Script review exercise

Objective. Find and fix four flaws in a provided script. **Difficulty.** [X]. **Prerequisites.** Section 47.9. **Environment. VM**, scratch folder.

Commands. Save as `report_bad.cmd`:

```
@echo off
set /p NAME=Customer name:
set /p FILE=Report file:
echo Customer: %NAME% >> "%TEMP%\cmdlab\report.log"
type "%TEMP%\cmdlab\data\%FILE%"
call :done %NAME%
exit /b
:done
echo finished for %1
```

Review it with the checklist and the two `findstr` searches, list every flaw, then write `report_good.cmd`.

Expected result. A list of flaws and a corrected script. **Explanation (key flaws).** (1) `NAME` is unvalidated and expanded into an `echo` line, so chaining and redirection are possible; (2) `FILE` is unvalidated, allowing traversal and wildcards; (3) `call :done %NAME%` re-expands and re-parses data; (4) no `setlocal`, so variables leak to the caller, and no error handling or exit codes. Secondary: bare `type` and `echo` rely on built-ins (fine), but any external tool call would trust PATH. **Verification.** The fixed script rejects hostile inputs from Labs 146-149 and writes only validated names to the log. **Troubleshooting.** If the allowlist rejects normal names with spaces, decide explicitly whether to allow spaces (and then keep them out of command lines or always quote). **Cleanup.** `del report_bad.cmd report_good.cmd`. **Challenge.** Add a regression test file of hostile inputs that must all be rejected. **Solution.** One input per line (`a & echo X`, `..\x`, `*`, `%COMPUTERNAME%`, `"x" & "`); run

the script against each and assert exit code 2.

Part XXI - Path and Command Resolution Security

Why "which program ran?" is a security question, and how to make the answer deterministic.

Chapter 48. Resolution Order and Hardening

48.1 The resolution pipeline, security view

```
User types:  tool
      |
  Doskey macro?  (interactive only)            --> a macro can shadow anything
      |
  Built-in command?  (cd, dir, echo, ...)      --> safe from file shadowing
      |
  Path or drive in the name?  (C:\x\tool, .\tool)  --> exact location, no search
      |
  Search: current directory FIRST, then each PATH folder in order
      |   in each folder: tool + each PATHEXT extension in order
      |   (.COM, .EXE, .BAT, .CMD, ... )
      v
  First match wins  --> CreateProcess  --> new process (the child parses its own arguments)
```

Security consequences:

Property	Risk	Control
Current directory searched first (for CMD name lookup)	A look-alike file in the folder you happen to be in runs instead of the system tool	`NoDefaultCurrentDirectoryInExePath=1`; `cd /d` to a trusted folder; absolute paths
PATH order	An earlier writable folder shadows later system folders	Keep system folders first; no user-writable folders in the system PATH
PATHEXT order	`tool.com` beats `tool.exe`; `.BAT` beats `.CMD`	Set `PATHEXT` explicitly in privileged scripts; avoid extension-less calls
Extension omission	`ping` could mean several files	Write the extension and path for important tools
Name collisions	Third-party installs (`find.exe` from Unix tools, `python.exe`, `curl.exe`) shadow Windows ones	`where` check; absolute paths for system tools
Built-ins first	A file named `dir.exe` is ignored by plain `dir`, but explicit `dir.exe` runs it	Do not rely on name alone
Doskey macros	Interactive shells can redefine commands	`doskey /macros` in suspicious sessions
Launching documents	`start file.ext` follows file associations in the registry	Audit associations (`assoc`, `ftype`, `HKCR`)

48.2 Verification habits

```
where tool
where /r "%SystemRoot%\System32" tool.exe
powershell -NoProfile -Command "(Get-Command tool).Source"
```

For a **running** process, verify the **image path** (`tasklist /v`, `wmic` replacement `Get-CimInstance Win32_Process`), the **signer**, and the **hash** (Chapter 26). For scripts: do not infer path from name.

48.3 Auditing the PATH for writable folders

A PATH folder that ordinary users can write to lets them plant look-alikes that privileged users and services may run. This audit script reports PATH entries whose ACL grants write-class rights to broad groups:

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
set /a HITS=0
for %%p in ("%PATH:;=" "%") do (
    set "D=%%~p"
    if defined D if exist "!D!\" (
        icacls "!D!" 2>nul | findstr /i /r /c:"Everyone:.*(F)" /c:"Everyone:.*(M)"
    .. /c:"Everyone:.*(W)" /c:"Users:.*(F)" /c:"Users:.*(M)" /c:"Users:.*(W)" >nul && (
            echo WRITABLE-BY-BROAD-GROUP: !D!
            set /a HITS+=1
        )
    )
)
echo PATH entries needing review: !HITS!
exit /b 0
```

Limitations (so you know what a "clean" result does and does not prove): the pattern recognizes simple permission letters (F, M, W) but not every combination of special rights (for example `WD`, `AD`); inherited entries and group nesting may need `Get-Acl` or Sysinternals `AccessChk` for exact answers; **writable parent folders** also matter (a user who can rename the folder can replace it). Treat findings as leads.

48.4 Hardening checklist

1. Keep `%SystemRoot%\System32` and `%SystemRoot%` first in the machine PATH; place third-party tools after them.
2. Install tools to protected locations (`Program Files`); never add `Downloads`, `Desktop`, `Temp`, `Users\Public` or any user-writable directory to the system PATH.
3. Set `NoDefaultCurrentDirectoryInExePath=1` machine-wide where compatible (test your applications) and in privileged scripts.
4. In privileged scripts: clean-room header (Section 47.7), absolute paths for system tools, explicit extensions.
5. Quote every path; avoid unquoted service and task action paths (Section 27.2).
6. Use application control (WDAC/AppLocker) so that executables outside trusted locations do not run.
7. Monitor: a system tool name executing from a non-system path; new executables in protected folders; PATH or `HKCU\Environment` changes.
8. Inventory: `where` for critical tool names on golden images; keep hashes of core tools.

48.5 A related class: DLL search order (concept)

Programs load DLLs with their own search order: known system DLLs, the application directory, the system folders, the current directory (depending on settings), then PATH. If an application directory or another searched location is **writable by a less-privileged user**, a planted DLL can be loaded by a more privileged instance of the program. Mitigations parallel the above: protected install folders, `SafeDllSearchMode` (on by default), signed binaries and application control. To investigate safely, use Sysinternals **Process Monitor** with filters `Result is NAME NOT FOUND` and `Path ends with .dll`, and look for privileged processes probing user-writable locations. Do not plant DLLs even in labs; observation is enough.

48.6 A safe demonstration of unquoted-path probing with CMD itself

CMD applies a similar "try progressively longer names" logic when you type an unquoted path with spaces. This harmless lab shows the principle without services:

```
md "C:\CmdLabU\Program Files\My App"
> "C:\CmdLabU\Program Files\My App\app.cmd" echo @echo REAL app ran
> "C:\CmdLabU\Program.cmd" echo @echo INTERCEPTED by shorter name Program.cmd
cmd /c C:\CmdLabU\Program Files\My App\app.cmd
```

Expected (verify on your build): the unquoted line runs `Program.cmd` (the shorter match) and prints `INTERCEPTED...`, instead of the real script. The quoted form `"C:\CmdLabU\Program Files\My App\app.cmd"` runs the intended file. Clean up with `rd /s /q C:\CmdLabU`. Services work with `CreateProcess` rules, which also probe progressively longer prefixes (`C:\Program.exe`, then `C:\Program Files\My.exe`, ...), so **unquoted service paths with spaces are a finding**; fix by quoting the `ImagePath`.

Security Labs

Labs 116-135. All use harmless artifacts, synthetic data or read-only audits. Run in a VM with a snapshot.

Chapter 49. Defensive Security Labs 116-135

SAFETY. Rules for every lab here: VM only; snapshot first; lab objects are named `CmdLab...`; nothing is downloaded; no lab runs an unknown binary; every lab that changes state has a rollback in Cleanup. Scripts that use PowerShell are fed through standard input (`type file.ps1 | powershell -NoProfile -Command -`), which works regardless of the script-file execution policy and keeps the lab files readable plain text. Execution policy is a safety feature for script files, not a security boundary.

Lab 116. Turn on command-line telemetry and read a 4688 event

Objective. Make process-creation events carry the command line, then find your own. **Difficulty.** [I]. **Prerequisites.** Section 44.3. **Environment. VM + snapshot**, elevated.

Commands.

```
auditpol /get /subcategory:"Process Creation" > ac_before.txt
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
    .. ProcessCreationIncludeCmdLine_Enabled
auditpol /set /subcategory:"Process Creation" /success:enable
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
    .. ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
cmd /c echo lab116 marker one two
wevtutil qe Security "/q:*[System[(EventID=4688)]]" /c:30 /rd:true /f:text | findstr /i
    .. /c:"lab116" /c:"Process Command Line"
```

Expected result. A 4688 event whose command line contains `lab116 marker one two`. **Explanation.** Without the registry setting the event exists but the command line is blank. **Verification.** Note the creator process and user fields. **Troubleshooting.** If nothing is found, check `type ac_before.txt` and re-run `auditpol /get`; Group Policy may be overriding local settings. **Cleanup.** If `ac_before.txt` shows `No Auditing`: `auditpol /set /subcategory:"Process Creation" /success:disable`. Then `reg delete "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v ProcessCreationIncludeCmdLine_Enabled /f` (if it did not exist before) and `del ac_before.txt`. **Challenge.** Explain the privacy trade-off. **Solution.** Command lines expose secrets and personal data; restrict log access and fix scripts that put secrets in arguments (Lab 123).

Lab 117. Masquerade detection: a system name in the wrong place

Objective. Detect a harmless renamed copy of a Microsoft binary by its path. **Difficulty.** [I]. **Prerequisites.** Section 45.2. **Environment. VM**, scratch folder.

Commands.

```
md masq
copy "%WINDIR%\System32\cmd.exe" masq\svchost.exe
start "MasqLab" masq\svchost.exe /k title MasqLab
powershell -NoProfile -Command "Get-CimInstance Win32_Process | Where-Object { $_.Name -eq
    .. 'svchost.exe' -and $_.ExecutablePath -ne ($env:SystemRoot + '\System32\svchost.exe') }
    .. | Select-Object ProcessId,ExecutablePath,CommandLine | Format-List"
```

Expected result. One process named `svchost.exe` whose path is your lab folder, plus its PID. **Explanation.** The name matches a system process; the path does not. This is the whole rule. **Verification.** Real `svchost.exe` instances do not appear in the filtered output. **Troubleshooting.** `ExecutablePath` is blank for some protected processes (not an issue for this filter). **Cleanup.** `taskkill /fi "windowtitle eq MasqLab*" /f` (or close the window), then `rd /s /q masq`. **Challenge.** Add parent-process output to the rule. **Solution.** Select `ParentProcessId` and look up the parent's name; real `svchost.exe` has parent `services.exe`.

Lab 118. Double extensions and the script host

Objective. Find double-extension files and understand what runs a script. **Difficulty.** [I]. **Prerequisites.** Section 45.4. **Environment. VM**, scratch folder.

Commands.

```
> invoice.pdf.cmd echo @echo This is a SCRIPT pretending to be a PDF
> notes.txt echo plain text
> hello.vbs echo WScript.Echo "hello from the script host"
> patterns.txt echo \.[a-z0-9][a-z0-9]*\.cmd$
>> patterns.txt echo \.[a-z0-9][a-z0-9]*\.bat$
>> patterns.txt echo \.[a-z0-9][a-z0-9]*\.vbs$
>> patterns.txt echo \.[a-z0-9][a-z0-9]*\.js$
>> patterns.txt echo \.[a-z0-9][a-z0-9]*\.hta$
>> patterns.txt echo \.[a-z0-9][a-z0-9]*\.scr$
dir /b | findstr /i /r /g:patterns.txt
assoc .vbs
cscript //nologo hello.vbs
reg query "HKLM\SOFTWARE\Microsoft\Windows Script Host\Settings" /v Enabled
```

Expected result. Only `invoice.pdf.cmd` is flagged; `.vbs` maps to `VBSFile`; the script prints its greeting; the `Enabled` query is usually "not found" (WSH enabled by default). **Explanation.** Explorer may display `invoice.pdf` when extensions are hidden; the rule keys on the **last** extension being executable and a second extension before it. **Verification.** Rename `notes.txt` to `notes.pdf.txt`; it is not flagged because the final extension is harmless. **Troubleshooting.** Quote carefully; `findstr /g` reads patterns from the file, one per line. **Cleanup.** `del invoice.pdf.cmd notes.txt hello.vbs patterns.txt`. **Challenge.** Design the Group Policy change that makes `.vbs` open in Notepad. **Solution.** Set the default association (file type handler) for script extensions to Notepad for standard users, or disable WSH with the `Enabled` value set to `0`; test for impact first.

Lab 119. PATHEXT ordering

Objective. Show that extension order decides which same-named file runs. **Difficulty.** [I]. **Prerequisites.** Section 12.1. **Environment.** Scratch folder.

Commands.

```
> dup.bat echo @echo I am dup.BAT
> dup.cmd echo @echo I am dup.CMD
echo %PATHEXT%
dup
set "OLDPE=%PATHEXT%"
set "PATHEXT=.COM;.EXE;.CMD;.BAT"
dup
set "PATHEXT=%OLDPE%"
where dup
```

Expected result. With the default order `dup` runs the `.BAT`; after reordering it runs the `.CMD`; the last command lists both files. **Explanation.** The first PATHEXT entry that matches wins. **Verification.** The two outputs differ. **Troubleshooting.** If both runs print the same file, check that you reassigned PATHEXT in the same window. **Cleanup.** `del dup.bat dup.cmd`; `PATHEXT` is restored by the last `set`. **Challenge.** Explain the detection angle. **Solution.** Two files with the same base name and different executable extensions in one folder are unusual; flag them.

Lab 120. Environment sanity check

Objective. Detect odd PATH and COMSPEC values. **Difficulty.** [I]. **Prerequisites.** Section 45.5. **Environment.** Scratch folder.

Commands. Save as `envcheck.cmd`:

```
@echo off
setlocal EnableExtensions
set "CS=%~1"
if not defined CS set "CS=%COMSPEC%"
if /i not "%CS%"=="%SystemRoot%\System32\cmd.exe" echo ALERT: COMSPEC is unusual: %CS%
for /f "tokens=1 delims=;" %%a in ("%PATH%") do (
    if /i not "%%a"=="%SystemRoot%\system32" echo NOTE: first PATH entry is %%a
)
reg query "HKCU\Environment" 2>nul | findstr /i "PATH COMSPEC PATHEXT" && echo ALERT:
    .. user-level PATH/COMSPEC/PATHEXT override present
echo checks complete
```

Run `envcheck`, then `envcheck C:\Users\Public\cmd.exe` (the argument simulates a bad value without changing your real COMSPEC).

Expected result. Normal run prints only `checks complete` (and possibly a `NOTE` if your PATH starts elsewhere); the simulated run prints the COMSPEC alert. **Explanation.** COMSPEC is the shell CMD launches for `cmd /c` internals; it must be the system CMD. **Verification.** Compare with `echo %COMSPEC%`. **Troubleshooting.** `Windows` capitalization differs; the `/i` flag handles it. **Cleanup.** `del envcheck.cmd`. **Challenge.** Add a check for `PATH` entries that no longer exist. **Solution.** Loop over entries with `if not exist "%%~p\" echo MISSING`.

Lab 121. Audit the PATH for writable folders

Objective. Run the Section 48.3 audit and understand its limits. **Difficulty.** [A]. **Prerequisites.** Section 48.3. **Environment. VM**, scratch folder, standard prompt.

Commands. Save the script from Section 48.3 as `pathaudit.cmd`. Run it. Then create a lab folder and put it in PATH for this session:

```
md C:\CmdLabPath
icacls C:\CmdLabPath /grant Users:(OI)(CI)M
set "PATH=C:\CmdLabPath;%PATH%"
pathaudit
```

Expected result. The first run reports zero or a few entries (investigate each); the second run flags `C:\CmdLabPath` (needs an elevated prompt to create the folder and grant rights). **Explanation.** A broad-write folder at the front of PATH is the dangerous configuration. **Verification.** `icacls C:\CmdLabPath`. **Troubleshooting.** Names such as `Users` are localized on non-English Windows. **Cleanup.** `rd /s /q C:\CmdLabPath`; close the window to discard the PATH change; `del pathaudit.cmd`. **Challenge.** List the limitations of the check. **Solution.** It misses special-rights combinations, inherited-versus-explicit distinctions, group nesting, and writable parent folders.

Lab 122. Unquoted path probing (CMD demonstration)

Objective. Show that an unquoted path with spaces can resolve to a shorter name. **Difficulty.** [I]. **Prerequisites.** Section 48.6. **Environment. VM**, elevated.

Commands.

```
md "C:\CmdLabU\Program Files\My App"
> "C:\CmdLabU\Program Files\My App\app.cmd" echo @echo REAL app ran
> "C:\CmdLabU\Program.cmd" echo @echo INTERCEPTED by shorter name
cmd /c C:\CmdLabU\Program Files\My App\app.cmd
cmd /c "C:\CmdLabU\Program Files\My App\app.cmd"
```

Expected result. The unquoted form prints `INTERCEPTED...`; the quoted form prints `REAL app ran`. **Explanation.** CMD tries the first token plus PATHEXT extensions before the rest. Windows service path parsing is a different mechanism with similar probing. **Verification.** Delete `Program.cmd` and re-run the unquoted line: it now fails or behaves differently. **Troubleshooting.** If both print `REAL`, record your build; behavior can differ with `cmd /c` quote handling. **Cleanup.** `rd /s /q C:\CmdLabU`. **Challenge.** Write an audit for unquoted service paths. **Solution.** Lab 88's PowerShell filter or Lab 130's script.

Lab 123. Secrets in command lines

Objective. Show that arguments are visible to other tools, and fix the pattern. **Difficulty.** [I]. **Prerequisites.** Technique 14. **Environment. VM**, two prompts. The "secret" is the fake string `FAKE-SECRET-123`.

Commands.

```
start "SecretLab" cmd /k "title SecretLab & rem password=FAKE-SECRET-123"
powershell -NoProfile -Command "Get-CimInstance Win32_Process | Where-Object { $_.CommandLine
    .. -match 'FAKE-SECRET' } | Select-Object ProcessId,Name,CommandLine | Format-List"
```

Now the safer pattern: a script that prompts for the value instead.

```
> askpw.cmd echo @echo off
>> askpw.cmd echo set /p "PW=Enter value: "
>> askpw.cmd echo echo value received, not printing it
start "AskLab" cmd /k askpw.cmd
powershell -NoProfile -Command "Get-CimInstance Win32_Process | Where-Object { $_.CommandLine
    .. -match 'askpw' } | Select-Object CommandLine"
```

Expected result. The first query shows the fake secret in plain text in the process list. The second shows only `askpw.cmd`, because the value was typed at a prompt and never appeared on a command line. **Explanation.** Any user on the machine (and every logging agent) can read command lines. **Verification.** If Lab 116 telemetry is still enabled, search 4688 for `FAKE-SECRET`. **Troubleshooting.** Typing the prompt value into the window is fine; do not type a real secret. **Cleanup.** Close both windows; `del askpw.cmd`. **Challenge.** Explain why environment variables are not a safe replacement. **Solution.** Child processes inherit them, and tools can read a process's environment; use a credential store, a protected file or stdin.

Lab 124. Hunt for secrets in logs

Objective. Scan synthetic logs for secret-looking patterns. **Difficulty.** [I]. **Prerequisites.** Technique 14. **Environment.** Scratch folder.

Commands.

```
> app.log echo 2026-10-01 10:00 INFO starting service
>> app.log echo 2026-10-01 10:01 DEBUG connecting with user=svc_backup password=FAKE-PASS-001
>> app.log echo 2026-10-01 10:02 INFO request ok
>> app.log echo 2026-10-01 10:03 DEBUG header Authorization: Bearer FAKE-TOKEN-XYZ
>> app.log echo 2026-10-01 10:04 ERROR cmd line was: tool.exe /apikey:FAKE-KEY-777 /run
> secrets.pat echo password=
>> secrets.pat echo pwd=
>> secrets.pat echo token
>> secrets.pat echo apikey
>> secrets.pat echo secret
>> secrets.pat echo Authorization:
findstr /i /n /g:secrets.pat app.log
findstr /i /v /g:secrets.pat app.log > app.clean.log
find /c /v "" app.clean.log
```

Expected result. Lines 2, 4 and 5 flagged; the clean file keeps lines 1 and 3 (2 lines). **Explanation.** Pattern scans are crude but effective first-pass checks. **Verification.** The count is 2. **Troubleshooting.** `findstr /g` treats each pattern as a regex; avoid special characters or add `/l`. **Cleanup.** `del app.log secrets.pat app.clean.log`. **Challenge.** Say what to do about real findings. **Solution.** Rotate the exposed secret, remove it at the source, purge or restrict the logs, and check where else it was used.

Lab 125. Command-line detection rules against synthetic telemetry

Objective. Write simple rules, measure hits and misses, and tune. **Difficulty.** [A]. **Prerequisites.** Chapters 44-46. **Environment.** Scratch folder. **Data is invented and harmless; nothing is executed.**

Commands.

```
> cmdlines.txt echo explorer.exe;cmd.exe /c dir C:\Users\bob\Documents
>> cmdlines.txt echo winword.exe;cmd.exe /c whoami
>> cmdlines.txt echo services.exe;svchost.exe -k netsvcs -p
>> cmdlines.txt echo explorer.exe;rundll32.exe shell32.dll,Control_RunDLL desk.cpl
>> cmdlines.txt echo winword.exe;rundll32.exe C:\Users\bob\AppData\Local\Temp\a1b2.dll,Start
>> cmdlines.txt echo outlook.exe;wscript.exe C:\Users\bob\Downloads\invoice.vbs
>> cmdlines.txt echo explorer.exe;mshta.exe C:\Users\bob\Downloads\form.hta
>> cmdlines.txt echo cmd.exe;certutil.exe -hashfile C:\Tools\setup.exe SHA256
>> cmdlines.txt echo cmd.exe;certutil.exe -encode C:\Users\bob\AppData\Local\Temp\in.dat
    .. C:\Users\bob\AppData\Local\Temp\out.txt
>> cmdlines.txt echo svchost.exe;rundll32.exe
>> cmdlines.txt echo sccm.exe;msiexec.exe /i C:\Windows\ccmcache\1\vendor.msi /qn
>> cmdlines.txt echo explorer.exe;msiexec.exe /i C:\Users\bob\Downloads\tool.msi /qn
>> cmdlines.txt echo taskeng.exe;cmd.exe /c C:\Scripts\backup.cmd
> rules.txt echo ^^winword\.exe;
>> rules.txt echo ^^outlook\.exe;
>> rules.txt echo ^^excel\.exe;
>> rules.txt echo rundll32\.exe$
>> rules.txt echo \\Temp\\.*\.dll
>> rules.txt echo \\Downloads\\.*\.vbs
>> rules.txt echo \\Downloads\\.*\.hta
>> rules.txt echo certutil\.exe -encode
>> rules.txt echo certutil\.exe -decode
findstr /i /n /r /g:rules.txt cmdlines.txt
```

Expected result. Hits on lines 2, 5, 6, 7, 9 and 10. **Explanation.** Rules combine parent, path and argument features. **Verification.** Line 8 (`certutil -hashfile`, benign) is correctly **not** flagged; line 12 (an MSI installed from `Downloads`) is **missed** (a false negative). **Troubleshooting.** CMD treats `^` as its escape character, so `echo ^^winword...` is needed to write a literal `^` (the regex start-of-line anchor) into the rules file. Check with `type rules.txt`. **Cleanup.** `del cmdlines.txt rules.txt`. **Challenge.** Add a rule for the missed line without flagging line 11. **Solution.** `msiexec\.exe .*\\Downloads\\` (path-based, so the software-distribution line from `ccmcache` stays clean). Record the trade-off: any rule trades coverage for noise.

Lab 126. Parent/child anomaly rules

Objective. Apply expected-lineage rules to synthetic process pairs. **Difficulty.** [I]. **Prerequisites.** Section 26.3. **Environment.** Scratch folder.

Commands.

```
> proc.csv echo services.exe,svchost.exe
>> proc.csv echo wininit.exe,services.exe
>> proc.csv echo wininit.exe,lsass.exe
>> proc.csv echo explorer.exe,cmd.exe
>> proc.csv echo winword.exe,cmd.exe
>> proc.csv echo cmd.exe,tasklist.exe
>> proc.csv echo explorer.exe,svchost.exe
>> proc.csv echo outlook.exe,wscript.exe
>> proc.csv echo cmd.exe,svchost.exe
echo --- svchost with the wrong parent
findstr /e /c:",svchost.exe" proc.csv | findstr /v /b /c:"services.exe,"
echo --- office parents
findstr /b /i /c:"winword.exe," /c:"excel.exe," /c:"outlook.exe," proc.csv
```

Expected result. The first rule prints `explorer.exe,svchost.exe` and `cmd.exe,svchost.exe`; the second prints `winword.exe,cmd.exe` and `outlook.exe,wscript.exe`. **Explanation.** Allowlisted parents for critical processes and blocklisted parents for interpreters are the two common rule families. **Verification.** Counts 2 and 2. **Troubleshooting.** `findstr /e` anchors at end of line. **Cleanup.** `del proc.csv`. **Challenge.** Why is `explorer.exe,cmd.exe` not flagged? **Solution.** It is the normal user-launched shell; flagging it would create noise without value.

Lab 127. Hash baseline and verification tool

Objective. Build `hashcheck.cmd` and detect a modified file. **Difficulty.** [A]. **Prerequisites.** Technique 12. **Environment.** Scratch folder.

Commands. Save as `hashcheck.cmd`:

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
set "DIR=%~2"
set "MAN=%~3"
if "%~1"=="" goto :usage
if "%~2"=="" goto :usage
if "%~3"=="" goto :usage
if /i "%~1"=="/make"   goto :make
if /i "%~1"=="/verify" goto :verify
:usage
echo Usage: %~nx0 /make folder manifest   or   %~nx0 /verify folder manifest
exit /b 2

:make
> "%MAN%" echo # sha256 manifest
for %%f in ("%DIR%\*") do (
    call :hash "%%~f"
    >> "%MAN%" echo !H!  %%~nxf
)
echo wrote %MAN%
exit /b 0

:verify
set /a BAD=0
for /f "usebackq eol=# tokens=1,*" %%a in ("%MAN%") do (
    if not exist "%DIR%\%%b" (
        echo MISSING %%b
        set /a BAD+=1
    ) else (
        call :hash "%DIR%\%%b"
        if /i "!H!"=="%%a" (echo OK      %%b) else (echo CHANGED %%b & set /a BAD+=1)
    )
)
echo mismatches: !BAD!
exit /b !BAD!

:hash
set "H="
for /f "skip=1 delims=" %%h in ('certutil -hashfile "%~1" SHA256') do if not defined H set
    .. "H=%%h"
set "H=!H: =!"
exit /b 0
```

Then:

```
md golden
copy "%WINDIR%\System32\ping.exe" golden
copy "%WINDIR%\System32\hostname.exe" golden
copy "%WINDIR%\System32\whoami.exe" golden
hashcheck /make golden golden.sha256
hashcheck /verify golden golden.sha256
echo x>> golden\hostname.exe
hashcheck /verify golden golden.sha256
echo %ERRORLEVEL%
```

Expected result. First verify: three `OK` lines, `mismatches: 0`. After appending a byte to the **copy**: `CHANGED hostname.exe`, `mismatches: 1`, exit code 1. **Explanation.** Any change flips the hash; a manifest stored somewhere the attacker cannot write turns this into integrity monitoring. **Verification.** Delete a file from `golden` to see `MISSING`. **Troubleshooting.** If every file shows CHANGED, check whether `certutil` prints spaces in the hash (the script removes them) and that the manifest lines contain two spaces. **Cleanup.** `rd /s /q golden & del golden.sha256 hashcheck.cmd`. **Challenge.** Explain where to store the manifest. **Solution.** On read-only or remote media, or signed; a manifest next to the files can be edited together with them.

Lab 128. Signature audit

Objective. Summarize signature status of a folder of executables. **Difficulty.** [I]. **Prerequisites.** Technique 12. **Environment.** Scratch folder.

Commands.

```
md sigtest
copy "%WINDIR%\System32\ping.exe" sigtest
> sigtest\unsigned.ps1 echo Write-Output 'unsigned script'
powershell -NoProfile -Command "Get-ChildItem sigtest -File | Get-AuthenticodeSignature |
    .. Select-Object @{n='File';e={Split-Path $_.Path
    .. -Leaf}},Status,@{n='Signer';e={$_.SignerCertificate.Subject}} | Format-Table -AutoSize"
powershell -NoProfile -Command "Get-ChildItem $env:ProgramFiles -Recurse -Include *.exe
    .. -ErrorAction SilentlyContinue | Select-Object -First 40 | Get-AuthenticodeSignature |
    .. Group-Object Status | Select-Object Count,Name"
```

Expected result. `ping.exe` shows `Valid` with a Microsoft signer (OS files are catalog-signed, and a copy still validates by hash); `unsigned.ps1` shows `NotSigned`; the second query summarizes the first 40 executables under Program Files by status. **Explanation.** Status plus signer plus context is the evaluation; `NotSigned` is not automatically bad. **Verification.** `Valid` count dominates on a typical system. **Troubleshooting.** Slow on large folders; keep the `-First 40`. **Cleanup.** `rd /s /q sigtest`. **Challenge.** Write the rule for "unsigned executable in a protected path that is newly created". **Solution.** Combine file creation time (`dir /t:c`) in the last day with `NotSigned` for files under `%ProgramFiles%` or `%SystemRoot%`; investigate each.

Lab 129. Scheduled-task risk audit

Objective. Flag tasks with risky characteristics. **Difficulty.** [A]. **Prerequisites.** Labs 99-100, technique 9. **Environment. VM**, elevated.

Commands. Save `taskaudit.ps1`:

```
$tasks = Get-ScheduledTask | Where-Object { $_.TaskPath -notlike '\Microsoft\*' }
$out = foreach ($t in $tasks) {
  foreach ($a in $t.Actions) {
    $cmd = ($a.Execute + ' ' + $a.Arguments).Trim()
    $flags = @()
    if ($cmd -match '\\Users\\|\\AppData\\|\\Temp\\|\\ProgramData\\|\\Downloads\\') { $flags
    .. += 'user-writable path' }
    if ($t.Principal.UserId -match 'SYSTEM') { $flags += 'runs as SYSTEM' }
    if ($t.Settings.Hidden) { $flags += 'hidden' }
    if ($cmd -match 'powershell|cmd\.exe|wscript|cscript|mshta|rundll32|regsvr32') { $flags +=
    .. 'interpreter' }
    if ($flags.Count -gt 0) {
      [pscustomobject]@{ Task = ($t.TaskPath + $t.TaskName); User = $t.Principal.UserId;
    .. Action = $cmd; Flags = ($flags -join ', ') }
    }
  }
}
$out | Format-List
```

Then:

```
schtasks /create /tn "CmdLab\Flagged" /tr "cmd /c echo benign >>
    .. C:\Users\Public\cmdlab_task.txt" /sc once /st 23:59 /f
schtasks /create /tn "CmdLab\Quiet" /tr "\"%WINDIR%\System32\ipconfig.exe\"" /sc once /st
    .. 23:58 /f
type taskaudit.ps1 | powershell -NoProfile -Command -
```

Expected result. `CmdLab\Flagged` listed with flags `user-writable path, interpreter`; `CmdLab\Quiet` not listed (plus any real non-Microsoft tasks on the VM). **Explanation.** Flags are leads, not verdicts. **Verification.** Also check the XML of each flagged task (`schtasks /query /tn ... /xml`). **Troubleshooting.** Quote nesting in `/tr` is delicate; keep the examples exactly. **Cleanup.** `schtasks /delete /tn "CmdLab\Flagged" /f & schtasks /delete /tn "CmdLab\Quiet" /f & del taskaudit.ps1`. **Challenge.** Add detection of tasks created in the last 7 days. **Solution.** Export the task XML (`Export-ScheduledTask -TaskName $t.TaskName -TaskPath $t.TaskPath`) and read its `<Date>` element, or use event 106 times.

Lab 130. Service risk audit

Objective. Find unquoted paths, missing binaries and writable folders. **Difficulty.** [A]. **Prerequisites.** Labs 87-88, technique 8. **Environment. VM**, elevated.

Commands. Save `svcaudit.ps1`:

```
function Get-ExePath($p) {
  if ($p -match '^\s*"([^"]+)"') { return $matches[1] }
  if ($p -match '^\s*([A-Za-z]:\\.*?\.exe)') { return $matches[1] }
  return ($p -split ' ')[0]
}
$rows = foreach ($s in Get-CimInstance Win32_Service) {
  if (-not $s.PathName) { continue }
  $exe = Get-ExePath $s.PathName
  $flags = @()
  if ($s.PathName -notmatch '^\s*"' -and $exe -match ' ') { $flags += 'unquoted path with
    .. spaces' }
  if (-not (Test-Path -LiteralPath $exe)) { $flags += 'binary missing' }
  else {
    try {
      $acl = Get-Acl -LiteralPath (Split-Path $exe)
      $w = $acl.Access | Where-Object { $_.AccessControlType -eq 'Allow' -and
    .. $_.IdentityReference.Value -match 'Everyone|BUILTIN\\Users|Authenticated Users' -and
    .. $_.FileSystemRights.ToString() -match 'Write|Modify|FullControl|CreateFiles|AppendData'
    .. }
      if ($w) { $flags += 'folder writable by broad group' }
    } catch { }
  }
  if ($exe -match '\\Users\\|\\Temp\\|\\AppData\\') { $flags += 'user-writable location' }
  if ($flags) { [pscustomobject]@{ Name = $s.Name; Account = $s.StartName; Path = $s.PathName;
    .. Flags = ($flags -join '; ') } }
}
$rows | Format-List
```

Then (never start the service):

```
sc create CmdLabSvc binPath= "C:\Users\Public\Cmd Lab Svc\svc.exe" start= demand
type svcaudit.ps1 | powershell -NoProfile -Command -
sc delete CmdLabSvc
```

Expected result. `CmdLabSvc` flagged for **unquoted path with spaces**, **binary missing** and **user-writable location**. Real services with findings need review. **Explanation.** Three independent conditions that make a service easy to hijack or already suspicious. **Verification.** Check `sc qc CmdLabSvc`. **Troubleshooting.** `Get-Acl` can be denied for protected folders; those are skipped. **Cleanup.** The delete line; `del svcaudit.ps1`. **Challenge.** Add the service account as a column and sort by privilege (LocalSystem first). **Solution.** Sort `$rows` by `Account -eq 'LocalSystem'`; a high-privilege service with a flag is the highest priority.

Lab 131. Autostart baseline and diff

Objective. Detect a new autostart-style entry with snapshots. **Difficulty.** [I]. **Prerequisites.** Section 30.7. **Environment.** Scratch folder, standard prompt. Uses a **harmless test key** (`HKCU\Software\CmdLab\RunSim`); no real autostart location is modified.

Commands. Save `autosnap.cmd`:

```
@echo off
> "%~1" echo ## autostart snapshot
for %%k in (
  "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
  "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
  "HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run"
  "HKCU\Software\CmdLab\RunSim"
) do (
    >> "%~1" echo [%%~k]
    reg query %%k 2>nul >> "%~1"
)
```

Then:

```
reg add HKCU\Software\CmdLab\RunSim /v Placeholder /t REG_SZ /d "nothing" /f
autosnap snap1.txt
reg add HKCU\Software\CmdLab\RunSim /v Updater /t REG_SZ /d "C:\Users\Public\updater.cmd" /f
autosnap snap2.txt
fc snap1.txt snap2.txt
```

Expected result. `fc` shows the `Updater` line added under `[HKCU\Software\CmdLab\RunSim]`. **Explanation.** The real Run keys are included in the snapshot so the same script works for monitoring; the test key makes the change safe to produce. **Verification.** Remove the value and re-snapshot: no differences. **Troubleshooting.** `reg query` prints `ERROR: ...` to stderr (hidden by `2>nul`) when a key does not exist. **Cleanup.** `reg delete HKCU\Software\CmdLab /f & del snap1.txt snap2.txt autosnap.cmd`. **Challenge.** Add the scheduled-task and service listings to the snapshot. **Solution.** Append `schtasks /query /fo csv` and `sc query state= all type= service` output, stripping time-varying columns before comparing.

Lab 132. Log-clearing detection (DANGER: VM only)

Objective. See what clearing a log looks like to a defender. **Difficulty.** [I]. **Prerequisites.** Technique list. **Environment. VM + snapshot only.** This lab clears the **Application** log of a disposable VM; do not do it anywhere else.

Commands.

```
eventcreate /t information /id 100 /l application /so CMDLAB /d "before clear"
wevtutil epl Application app_before.evtx
wevtutil gli Application | findstr /i "numberOfLogRecords oldestRecordNumber"
wevtutil cl Application
wevtutil gli Application | findstr /i "numberOfLogRecords oldestRecordNumber"
wevtutil qe System "/q:*[System[(EventID=104)]]" /c:1 /rd:true /f:text
```

Expected result. The record counts drop to nearly zero; the System log contains event **104** (the Application log was cleared) naming the account that did it; your exported copy preserves the old events. **Explanation.** Clearing is **detectable**, and defenders (a) alert on 104 and Security 1102, (b) forward logs off-box so a local clear cannot erase history, (c) look for record-number gaps. **Verification.** `wevtutil qe app_before.evtx /lf:true /c:3 /f:text` still shows the earlier events. **Troubleshooting.** Needs elevation. **Cleanup.** `reg delete HKLM\SYSTEM\CurrentControlSet\Services\EventLog\Application\CMDLAB /f`; `del app_before.evtx` (or revert the VM snapshot). **Challenge.** Write the alert rule. **Solution.** Alert when 1102 (Security) or 104 (System) appears and the account is not the approved log-maintenance identity; page a human immediately.

Lab 133. Inventory of dual-use configuration (read-only)

Objective. Check BITS jobs, WMI event subscriptions, Windows Installer elevation policy and script-host settings. **Difficulty.** [I]. **Prerequisites.** Chapter 46. **Environment.** Elevated prompt.

Commands.

```
bitsadmin /list /allusers
powershell -NoProfile -Command "Get-CimInstance -Namespace root\subscription -ClassName
    .. __EventFilter | Select-Object Name; Get-CimInstance -Namespace root\subscription
    .. -ClassName __EventConsumer | Select-Object Name"
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer" /v AlwaysInstallElevated
reg query "HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer" /v AlwaysInstallElevated
reg query "HKLM\SOFTWARE\Microsoft\Windows Script Host\Settings" /v Enabled
assoc .hta
assoc .js
```

Expected result. Typically no BITS jobs; WMI shows only default Microsoft entries (for example `SCM Event Log Filter`/`Consumer`); both `AlwaysInstallElevated` queries report **not found** (good: the policy is not enabled); the script-host key is not found (WSH enabled by default). **Explanation.** `AlwaysInstallElevated` set to 1 in **both** hives lets any user install packages with SYSTEM rights, which is a well-known privilege-escalation misconfiguration, so it must stay unset. Unexpected WMI subscriptions or BITS jobs with notify commands deserve investigation. **Verification.** Record each result in a table. **Troubleshooting.** `bitsadmin` is deprecated but present; `Get-BitsTransfer -AllUsers` is the alternative. **Cleanup.** None. **Challenge.** Say what you would do if `AlwaysInstallElevated` were `0x1`. **Solution.** Treat it as a finding, check who set it and when (Group Policy, registry write events), reset it, and review recent installs (MsiInstaller events).

Lab 134. Configuration drift monitor

Objective. Snapshot key security settings and detect change. **Difficulty.** [A]. **Prerequisites.** Labs 107, 131. **Environment. VM**, elevated.

Commands. Save `drift.cmd`:

```
@echo off
setlocal EnableExtensions
if /i "%~1"=="/base" (set "OUT=%~dp0drift_base.txt") else (set "OUT=%~dp0drift_now.txt")
> "%OUT%" echo == firewall
netsh advfirewall show allprofiles state >> "%OUT%"
>> "%OUT%" echo == uac
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" /v EnableLUA >>
    .. "%OUT%"
>> "%OUT%" echo == administrators
net localgroup Administrators >> "%OUT%"
>> "%OUT%" echo == defender
powershell -NoProfile -Command "$s=Get-MpComputerStatus;
    .. 'RealTime='+$s.RealTimeProtectionEnabled; 'Tamper='+$s.IsTamperProtected" >> "%OUT%"
>> "%OUT%" echo == listeners
for /f "tokens=2" %%a in ('netstat -ano ^| findstr LISTENING ^| sort') do >> "%OUT%" echo %%a
if /i "%~1"=="/base" (echo baseline saved & exit /b 0)
fc "%~dp0drift_base.txt" "%OUT%" >nul && (echo NO DRIFT & exit /b 0)
echo DRIFT DETECTED:
fc "%~dp0drift_base.txt" "%OUT%"
exit /b 1
```

Then:

```
drift /base
drift
powershell -NoProfile -Command
    .. "$l=[Net.Sockets.TcpListener]::new([Net.IPAddress]::Loopback,5599); $l.Start();
    .. Start-Sleep 20"
```

(Run the third command in a second window, then run `drift` again within 20 seconds.)

Expected result. `NO DRIFT` first; `DRIFT DETECTED` with the extra `127.0.0.1:5599` listener afterwards. **Explanation.** Drift monitoring turns the audit commands into a change detector. **Verification.** After the listener exits, `drift` reports no drift again. **Troubleshooting.** Ephemeral listeners vary on busy systems, so a few unrelated listener changes may appear; compare carefully. **Cleanup.** `del drift.cmd drift_base.txt drift_now.txt`. **Challenge.** Add the count of enabled inbound firewall rules. **Solution.** Append `netsh advfirewall firewall show rule name=all dir=in status=enabled | find /c "Rule Name"` to the snapshot.

Lab 135. Security review report (capstone of Part XIX)

Objective. Combine the audits into one report with a summary. **Difficulty.** [A]. **Prerequisites.** Labs 121, 129, 130, 134. **Environment. VM**, elevated, folder containing `pathaudit.cmd`, `svcaudit.ps1`, `taskaudit.ps1`, `drift.cmd` with a baseline.

Commands. Save as `secreview.cmd`:

```
@echo off
setlocal EnableExtensions
set "R=%~dp0secreview.txt"
> "%R%" echo Security review of %COMPUTERNAME%  %DATE% %TIME%
>> "%R%" echo.
>> "%R%" echo ===== PATH folders writable by broad groups
call "%~dp0pathaudit.cmd" >> "%R%"
>> "%R%" echo ===== Service findings
type "%~dp0svcaudit.ps1" | powershell -NoProfile -Command - >> "%R%"
>> "%R%" echo ===== Scheduled task findings
type "%~dp0taskaudit.ps1" | powershell -NoProfile -Command - >> "%R%"
>> "%R%" echo ===== Configuration drift
call "%~dp0drift.cmd" >> "%R%"
set /a P=0, S=0, T=0
for /f %%n in ('findstr /c:"WRITABLE-BY-BROAD-GROUP" "%R%" ^| find /c /v ""') do set P=%%n
for /f %%n in ('findstr /b /c:"Flags" "%R%" ^| find /c /v ""') do set /a S=%%n
>> "%R%" echo.
>> "%R%" echo ===== SUMMARY
>> "%R%" echo Writable PATH entries : %P%
>> "%R%" echo Flagged items (services + tasks) : %S%
findstr /c:"DRIFT DETECTED" "%R%" >nul && (>> "%R%" echo Drift : YES) || (>> "%R%" echo Drift
    .. : none)
type "%R%" | findstr /b /c:"Writable" /c:"Flagged" /c:"Drift :"
exit /b 0
```

Run it twice: once on the clean VM and once after recreating the flagged lab objects from Labs 121, 129 and 130.

Expected result. A single report file and a three-line summary that changes between the clean and the "dirty" run. **Explanation.** The point is repeatability: the same checks, the same format, comparable results over time. **Verification.** Compare summaries; open the report and confirm each finding is explained. **Troubleshooting.** If a sub-script is missing the section is empty; recreate it from its lab. **Cleanup.** Delete the lab objects as each lab instructs, then the report files. **Challenge.** Rank findings High/Medium/Low. **Solution.** High: broad-write PATH folder or service binary folder with a SYSTEM service; Medium: unquoted path, task with interpreter in a user-writable path; Low: informational drift. Always justify each rating in the report.

Part XXII - Incident Response

A repeatable method for handling suspicious activity on a Windows host, practiced on a completely benign simulated incident with a full solution.

Chapter 50. Incident Response Method

50.1 What incident response is (and is not)

Incident response (IR) is the disciplined handling of a suspected security event: decide whether it is real, understand its scope, stop it from spreading or continuing, remove it, restore normal operation and learn from it. It is not "run an antivirus and reboot". Hasty cleanup destroys evidence, hides the entry point, and often leaves persistence behind.

The widely used lifecycle (NIST SP 800-61):

Phase	Goal	Typical CMD-level work
Preparation	Be able to respond	Baselines, logging enabled, trusted toolkit on external media, contact list
Detection and analysis	Is it real? What happened? How far did it go?	Triage, evidence collection, timeline, scoping
Containment	Limit damage without destroying evidence	Disable (not delete) tasks/accounts, isolate network, block
Eradication	Remove the cause and persistence	Delete task, service, registry entries, files; fix the entry point
Recovery	Return to normal safely	Re-enable services, restore from known-good, monitor
Post-incident	Learn	Report, lessons learned, improve detections

50.2 Ground rules ("do no harm")

1. **Write down everything**: commands run, times, who ran them, and results. Your own actions will appear in the evidence.
2. **Preserve before you change.** Collect volatile data first, copy artifacts before editing, export logs before anything clears them.
3. **Prefer reversible containment** (disable, isolate) over irreversible actions (delete, wipe) until the evidence is safely stored.
4. **Do not execute suspicious files** on a production machine. Read them as text (`type`, a viewer), hash them, and analyze in an isolated lab.
5. **Use trusted tools**: run binaries from known-good external media or by absolute path from `System32`; do not run tools from the suspect's folders.
6. **Be careful with reboots and shutdowns.** A reboot erases volatile evidence (processes, connections, DNS cache, memory).
7. **Work from copies.** Analyze evidence copies, not originals, where practical.
8. **Escalate and communicate** according to your organization's policy; legal and privacy rules may apply to evidence handling.

50.3 Evidence handling

- **Chain of custody**: for each item record what it is, where and when it was collected, by whom, how (command), its **hash**, and who held it afterward.

- **Hashes**: compute SHA-256 at collection and again before analysis; mismatches mean the evidence changed.
- **Storage**: write to external or network media, not to the suspect disk (writing to the disk can overwrite deleted data).
- **Time**: record the system's time zone (`tzutil /g`) and any clock offset against a trusted source. Event log XML timestamps are **UTC**.

50.4 Questions and where the answers live

Question	Evidence	Typical command or place
Who executed it?	Process creation events, file owner, task author, logon events	4688 (`Subject`), `dir /q`, task XML `Author`, 4624
When?	File timestamps, event times, task run times	`dir /t:c /t:w`, `wevtutil qe`, `schtasks /query /v`
What commands ran?	Command-line telemetry, script contents, PowerShell logs, history	4688 `CommandLine`, Sysmon 1, `type script`, `ConsoleHost_history.txt`
What processes were created?	Process telemetry, Prefetch, current process list	4688, `tasklist`, CIM process list, `%WINDIR%\Prefetch`
What files changed?	Timestamps, USN journal, file system listings, hashes	`dir /s /a /t:w`, `fsutil usn readjournal`
Is there a scheduled task?	Task store, TaskCache, logs	`schtasks /query /v`, `%WINDIR%\System32\Tasks`, 106/140/141/200/201
Did a service change?	Services registry, logs	`sc query`, `sc qc`, `reg query ...Services`, 7045/4697/7040
Was there network activity?	Connections, DNS cache, firewall log, proxy/DNS logs, Sysmon 3	`netstat -ano`, `ipconfig /displaydns`, `pfirewall.log`, 5156
What evidence remains?	Everything above, plus what is **absent**	Document logging gaps explicitly
How do we remediate?	Findings	Section 50.6
Is the cleanup complete?	Re-run the same detections	Section 50.7

Absence of evidence is not evidence of absence. If network auditing was never enabled, "no connection recorded" means "unknown". State your confidence.

50.5 Scoping and timeline building

Build a **timeline**: one table with *time (UTC), source, event, significance*. Merge file system times, event log entries, task and service creation times and process events. Sort by time. The pattern (first suspicious activity, persistence installation, repeated execution, last activity) often shows the entry point and the dwell time. Then **scope**: which other hosts, accounts and files show the same indicators (hashes, names, paths, task names, destination addresses)?

50.6 Containment, eradication, recovery (order of operations)

```
1. PRESERVE   export task XML, sc qc output, registry keys, copy files, export logs, hash
   .. everything
2. CONTAIN    disable the task, set the service to disabled, isolate the machine if needed
3. ERADICATE  delete task, service, registry entries, files (in reverse order of dependency)
4. FIX CAUSE  close the entry point: weak ACLs, exposed credentials, missing patch, user action
5. RECOVER    restore services and settings from known-good; re-enable monitoring
6. VERIFY     Section 50.7
```

If **credentials may have been exposed**, rotate them. If the **host's integrity is in doubt** (for example evidence of privileged tampering), rebuilding from known-good media is often safer than cleaning.

50.7 Verifying cleanup

Re-run every detection you used to find the problem and expect "not found": task query, service query, registry query, file listing, process list, startup locations, listening sockets. Compare to a baseline if one exists. **Monitor** the host for recurrence (new task/service events, the same file names or hashes) for a defined period.

50.8 Reporting template

```
Incident ID / host / analyst / date
Summary (3 sentences)
Timeline (UTC)
Findings: persistence, execution, network, files, accounts
Evidence list with hashes and storage location
Impact and confidence level (and gaps in logging)
Actions taken (time, command, result)
Root cause / entry point
Recommendations (detections, hardening, process)
```

Part XXIII - Windows Forensics (Command-Line Collection)

What to collect, in what order, with what tools, and how to keep the collection defensible.

Chapter 51. Collecting and Handling Evidence

51.1 Volatile versus persistent evidence

Order of volatility: collect the data that disappears fastest first.

Evidence	Volatility	Commands (absolute paths from `System32` recommended)	Notes
Date, time, time zone	Immediate	`echo %DATE% %TIME%`, `tzutil /g`	Anchor for all other times
Network connections and listeners	Seconds	`netstat -ano`, `netstat -s`	Run before anything else that touches the network
ARP cache, DNS cache, routes	Minutes	`arp -a`, `ipconfig /displaydns`, `route print`	Show recent peers and names
Running processes with paths and command lines	Until exit	`tasklist /v /fo csv`, `tasklist /svc`, CIM process list	Parent/child links and creation times
Logged-on users and sessions	Until logoff	`quser`, `net session`, `net use`	`quser` is absent on some editions
Loaded modules, open files	Until exit	`tasklist /m`, `openfiles` (needs the global flag enabled)	Optional and heavy
Environment, system info	Stable	`set`, `systeminfo`, `ver`	
Services, scheduled tasks	Persistent	`sc query state= all`, `sc qc`, `schtasks /query /fo csv /v`	Persistence mechanisms
Local users and groups, shares	Persistent	`net user`, `net localgroup Administrators`, `net share`	
Firewall and Defender state	Persistent	`netsh advfirewall show allprofiles`, `Get-MpComputerStatus`	Also exclusions
Event logs	Persistent but rotating	`wevtutil epl <log> file.evtx`	Export early; logs roll over
Registry (selected keys)	Persistent	`reg export KEY file.reg`	Autostart, services, environment
File listings with times	Persistent	`dir /s /a /t:c`, `/t:w`, `/t:a`	Suspect folders first
Prefetch, Amcache, ShimCache, LNK, Jump Lists	Persistent	copy `%WINDIR%\Prefetch\*.pf` (if enabled); other artifacts need parsers	Show program execution history
USN journal	Persistent but wraps	`fsutil usn readjournal C: csv` (admin; large)	File create/rename/delete history
PowerShell history	Persistent	`%APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`	Per user
Memory	Most volatile	Dedicated imaging tools (outside this book's scope)	Capture before shutdown if policy requires

Specialized parsers (for example Eric Zimmerman's tools, KAPE, Autopsy, Volatility) are used for Prefetch, Amcache, MFT, registry hives and memory; CMD's job is to **collect safely and quickly**.

> **DANGER / CAUTION.** This book does **not** collect the `SAM` or `SECURITY` hives or process memory for credential material. If your organization's forensic policy requires them, follow that policy and its legal controls. Never extract or use credentials from a system you are not authorized to examine.

51.2 Trusted tools and a small footprint

- Run from external, write-protected media when possible; verify the tool hashes before collection.
- Call tools by **absolute path** (`%SystemRoot%\System32\...`); on 64-bit Windows run a **64-bit** CMD (a 32-bit CMD sees `SysWOW64` instead of `System32`).
- Expect your commands to be recorded in telemetry and to leave artifacts (Prefetch entries, new processes); log them.
- Write output to external media. If you must write locally, use a dedicated folder and accept that it changes the disk.

51.3 A volatile-evidence collector

Save as `collect_volatile.cmd`, run **elevated** from trusted media: `collect_volatile.cmd E:\evidence\HOST01`.

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem collect_volatile.cmd - order-of-volatility collector with hash manifest
if "%~1"=="" (echo Usage: %~nx0 output_folder & exit /b 2)
fltmc >nul 2>&1 || echo WARNING: not elevated - some data will be incomplete
set "OUT=%~1"
md "%OUT%" 2>nul
if not exist "%OUT%\" (echo ERROR: cannot create "%OUT%" & exit /b 1)
set "SYS=%SystemRoot%\System32"
set "LOG=%OUT%\collection_log.txt"
> "%LOG%" echo Collection started %DATE% %TIME% on %COMPUTERNAME% by %USERDOMAIN%\%USERNAME%

call :step 01_datetime
> "%OUT%\01_datetime.txt" echo %DATE% %TIME%
"%SYS%\tzutil.exe" /g >> "%OUT%\01_datetime.txt" 2>&1

call :step 02_netstat
"%SYS%\NETSTAT.EXE" -ano > "%OUT%\02_netstat.txt" 2>&1
call :step 03_arp
"%SYS%\ARP.EXE" -a > "%OUT%\03_arp.txt" 2>&1
call :step 04_dnscache
"%SYS%\ipconfig.exe" /displaydns > "%OUT%\04_dnscache.txt" 2>&1
call :step 05_ipconfig
"%SYS%\ipconfig.exe" /all > "%OUT%\05_ipconfig.txt" 2>&1
call :step 06_routes
"%SYS%\ROUTE.EXE" print > "%OUT%\06_routes.txt" 2>&1
call :step 07_tasklist
"%SYS%\tasklist.exe" /v /fo csv > "%OUT%\07_tasklist.csv" 2>&1
"%SYS%\tasklist.exe" /svc /fo csv > "%OUT%\07_tasklist_svc.csv" 2>&1
call :step 08_process_details
"%SYS%\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -Command "Get-CimInstance
    .. Win32_Process | Select-Object
    .. ProcessId,ParentProcessId,Name,ExecutablePath,CommandLine,CreationDate | ConvertTo-Csv
    .. -NoTypeInformation" > "%OUT%\08_process_details.csv" 2>&1
call :step 09_sessions
"%SYS%\quser.exe" > "%OUT%\09_quser.txt" 2>&1
"%SYS%\net.exe" session > "%OUT%\09_net_session.txt" 2>&1
"%SYS%\net.exe" use > "%OUT%\09_net_use.txt" 2>&1
call :step 10_services
"%SYS%\sc.exe" query state= all type= service > "%OUT%\10_services.txt" 2>&1
call :step 11_tasks
"%SYS%\schtasks.exe" /query /fo csv /v > "%OUT%\11_tasks.csv" 2>&1
call :step 12_accounts
"%SYS%\net.exe" user > "%OUT%\12_users.txt" 2>&1
"%SYS%\net.exe" localgroup Administrators > "%OUT%\12_admins.txt" 2>&1
"%SYS%\net.exe" share > "%OUT%\12_shares.txt" 2>&1
call :step 13_system
"%SYS%\systeminfo.exe" > "%OUT%\13_systeminfo.txt" 2>&1
set > "%OUT%\13_environment.txt"
call :step 14_firewall
"%SYS%\netsh.exe" advfirewall show allprofiles > "%OUT%\14_firewall.txt" 2>&1
call :step 15_autostart_registry
for %%k in ("HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
    .. "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
    .. "HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run" "HKCU\Environment") do
    .. (
    >> "%OUT%\15_autostart_registry.txt" echo [%%~k]
    "%SYS%\reg.exe" query %%k >> "%OUT%\15_autostart_registry.txt" 2>&1
```

```
)
call :step 16_eventlogs
for %%l in (Security System Application Microsoft-Windows-TaskScheduler/Operational
    .. "Microsoft-Windows-Windows Defender/Operational") do (
    set "LN=%%~l"
    set "FN=!LN:/=_!"
    "%SYS%\wevtutil.exe" epl "%%~l" "%OUT%\16_!FN!.evtx" /ow:true >> "%LOG%" 2>&1
)

call :step hashing
> "%OUT%\MANIFEST_sha256.txt" echo # sha256  file
for %%f in ("%OUT%\*") do (
    if /i not "%%~nxf"=="MANIFEST_sha256.txt" (
        call :hash "%%~f"
        >> "%OUT%\MANIFEST_sha256.txt" echo !H!  %%~nxf
    )
)
>> "%LOG%" echo Collection finished %DATE% %TIME%
echo Done. Evidence in "%OUT%". Store MANIFEST_sha256.txt separately.
exit /b 0

:step
>> "%LOG%" echo %TIME%  %~1
echo [%TIME%] %~1
exit /b 0

:hash
set "H="
for /f "skip=1 delims=" %%h in ('certutil -hashfile "%~1" SHA256') do if not defined H set
    .. "H=%%h"
set "H=!H: =!"
exit /b 0
```

Walkthrough points: output numbering preserves the **order of volatility**; every tool is invoked by absolute path; errors are captured with `2>&1` so a failed `quser` is itself evidence of the edition; the log file records start and finish times and each step; the hash manifest is written last (and excludes itself); event logs are exported as `.evtx` for offline analysis (the `/` in a channel name is replaced for the file name). Limitations: `net session` and `quser` may fail on some editions, `systeminfo` is slow and runs late for that reason, and the collector itself changes the system (new processes, Prefetch entries, the log folder).

51.4 Reading the evidence: what each artifact tells you

Artifact	Questions it answers	Typical caveat
`netstat -ano`	Which processes listen or talk now	Point in time; short connections missing
DNS cache	Recently resolved names	Cleared by reboot, flush or TTL
Process CSV with parents and creation dates	Lineage, command lines	PIDs are reused; compare creation dates
Services/tasks	Persistence	Normal software also persists
Event logs	Who, when, what changed	Only what was audited
Registry autostart keys	Startup persistence	HKCU entries per user
File timestamps	When files were created/changed	Timestamps can be altered; copy and extraction preserve or change them differently

51.5 Validating a collection

Before analysis: verify the hash manifest, confirm every expected file exists and is non-empty (or contains an explained error), confirm the clock/time zone was recorded, and record who collected, from which media, with which tool hashes.

Chapter 52. Simulated Incident and IR Labs

52.1 The simulation (benign)

> **SAFETY.** The simulation creates only harmless objects with the name `CmdLabIR`: a hidden script that appends a heartbeat line to a log and pings loopback; a scheduled task that runs it; a service **entry** that is never started; a marker in `HKCU\Software\CmdLabIR`; an alternate data stream. No network traffic leaves the machine. It runs only in a **VM with a snapshot**, from an **elevated** prompt, and is removed by a teardown script.

Scenario. *An endpoint alert reports that* `cmd.exe` *ran a script from* `C:\Users\Public` *on this workstation. The user says they did not install anything. You are the responder. Determine: who executed it; when; what commands ran; which processes were created; which files changed; whether a scheduled task exists; whether a service changed; whether network activity occurred; what evidence remains; how to remediate; and how to verify cleanup.*

Spoiler warning. The setup script below reveals everything the simulation creates. **Run it without reading it closely**, work the labs, and only then compare with the setup source and the solutions.

Save as `irsim_setup.cmd`:

```
@echo off
rem IRSIM setup - BENIGN SIMULATION. VM ONLY. Run from an ELEVATED prompt.
setlocal EnableExtensions
fltmc >nul 2>&1 || (echo Run this from an elevated prompt & exit /b 4)
set "BASE=C:\Users\Public\CmdLabIR"
set "STATE=%ProgramData%\CmdLabIR_state"
md "%STATE%" 2>nul

rem remember prior settings so the teardown can restore them
auditpol /get /subcategory:"Process Creation" > "%STATE%\audit_prior.txt"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
    .. ProcessCreationIncludeCmdLine_Enabled >nul 2>&1 && (> "%STATE%\cmdline_existed.txt"
    .. echo yes)
wevtutil gl Microsoft-Windows-TaskScheduler/Operational | findstr /i /c:"enabled: true" >nul
    .. && (> "%STATE%\tshist_enabled.txt" echo yes)

rem make sure evidence will exist
auditpol /set /subcategory:"Process Creation" /success:enable >nul
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
    .. ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f >nul
wevtutil sl Microsoft-Windows-TaskScheduler/Operational /e:true

rem the "suspicious" script: logs a heartbeat and pings loopback only
md "%BASE%" 2>nul
> "%BASE%\updater.cmd" echo @echo off
>> "%BASE%\updater.cmd" echo echo %%DATE%% %%TIME%% heartbeat user=%%USERNAME%% ^>^>
    .. "%BASE%\heartbeat.log"
>> "%BASE%\updater.cmd" echo ping -n 1 127.0.0.1 ^>nul
> "%BASE%\updater.cmd:origin" echo ZoneId=3 simulated-origin-downloaded
attrib +h "%BASE%\updater.cmd"

rem persistence-style objects (benign, pointing at the harmless script)
schtasks /create /tn "CmdLabIR\SystemUpdateCheck" /tr "\"%BASE%\updater.cmd\"" /sc once /st
    .. 23:59 /f >nul
sc create CmdLabIRSvc binPath= "C:\Users\Public\CmdLabIR\Update Service\svc.exe" start= demand
    .. DisplayName= "Windows Update Helper" >nul
reg add HKCU\Software\CmdLabIR /v Script /t REG_SZ /d "%BASE%\updater.cmd" /f >nul

rem execute it twice: once by hand, once through the task
cmd /c ""%BASE%\updater.cmd""
schtasks /run /tn "CmdLabIR\SystemUpdateCheck" >nul
timeout /t 5 /nobreak >nul
echo Simulation created. Close this window and begin Lab 136.
exit /b 0
```

Save as `irsim_teardown.cmd`:

```
@echo off
rem IRSIM teardown - removes everything the simulation created and restores settings.
setlocal EnableExtensions
fltmc >nul 2>&1 || (echo Run this from an elevated prompt & exit /b 4)
set "BASE=C:\Users\Public\CmdLabIR"
set "STATE=%ProgramData%\CmdLabIR_state"
schtasks /delete /tn "CmdLabIR\SystemUpdateCheck" /f >nul 2>&1
sc delete CmdLabIRSvc >nul 2>&1
reg delete HKCU\Software\CmdLabIR /f >nul 2>&1
attrib -h "%BASE%\updater.cmd" >nul 2>&1
rd /s /q "%BASE%" 2>nul
if not exist "%STATE%\tshist_enabled.txt" wevtutil sl
    .. Microsoft-Windows-TaskScheduler/Operational /e:false
if not exist "%STATE%\cmdline_existed.txt" reg delete
    .. "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
    .. ProcessCreationIncludeCmdLine_Enabled /f >nul 2>&1
findstr /i /c:"No Auditing" "%STATE%\audit_prior.txt" >nul && auditpol /set
    .. /subcategory:"Process Creation" /success:disable >nul
rd /s /q "%STATE%" 2>nul
echo Simulation removed. Verify with: schtasks /query /tn "CmdLabIR\SystemUpdateCheck"  and
    .. sc query CmdLabIRSvc
exit /b 0
```

Lab 136. Set up the simulated incident

Objective. Create the scenario and record your starting state. **Difficulty.** [B]. **Prerequisites.** Chapter 50. **Environment. VM + snapshot**, elevated prompt.

Commands.

```
echo %DATE% %TIME% > lab136_start.txt
irsim_setup.cmd
echo %DATE% %TIME% >> lab136_start.txt
type lab136_start.txt
```

Expected result. `Simulation created.` and two timestamps bracketing the creation (your "alert window"). **Explanation.** Real incidents have no such bracket; here it lets you check your timeline later. **Verification.** Do **not** list the simulation's objects yet. **Troubleshooting.** If setup reports not elevated, reopen an Administrator prompt. **Cleanup.** At the end of Lab 145 run `irsim_teardown.cmd`. **Challenge.** Write down, before investigating, what you expect to find. **Solution.** Predictions are only a hypothesis; the labs test them.

Lab 137. Triage: find the artifact, owner and times

Objective. Locate the script and answer who and when from the file system. **Difficulty.** [I]. **Prerequisites.** Lab 136. **Environment. VM**, elevated.

Commands.

```
dir /s /b /a C:\Users\Public\*.cmd
dir /a /q /t:c "C:\Users\Public\CmdLabIR"
dir /a /q /t:w "C:\Users\Public\CmdLabIR"
dir /a /r "C:\Users\Public\CmdLabIR"
attrib "C:\Users\Public\CmdLabIR\updater.cmd"
icacls "C:\Users\Public\CmdLabIR"
more < "C:\Users\Public\CmdLabIR\updater.cmd:origin"
```

Expected result. `updater.cmd` is found only because of `/a` (hidden); `dir /q` shows the **owner** (the administrator account that ran setup); creation time is the setup time; `dir /r` lists the stream `updater.cmd:origin:$DATA`; `attrib` shows `H`; `icacls` shows Users inherit modify from `C:\Users\Public`; the stream says `ZoneId=3 simulated-origin-downloaded`. **Explanation.** Hidden attribute and an alternate stream are both obscurity, not security; Public is writable by everyone. **Verification.** Note creation time T1 and last-write times for `updater.cmd` and `heartbeat.log`. **Troubleshooting.** If nothing is found, check you used `/a`. **Cleanup.** None (read-only). **Challenge.** Explain why the owner is a weak indicator of "who executed it". **Solution.** Ownership shows who created the file, not who ran it; creation may be by an admin, an installer or a tool. Use process and logon events for execution.

Lab 138. Static analysis: read the script without running it

Objective. Determine what the script does using text only. **Difficulty.** [I]. **Prerequisites.** Lab 137. **Environment. VM**.

Commands.

```
type "C:\Users\Public\CmdLabIR\updater.cmd"
type "C:\Users\Public\CmdLabIR\heartbeat.log"
find /c /v "" "C:\Users\Public\CmdLabIR\heartbeat.log"
certutil -hashfile "C:\Users\Public\CmdLabIR\updater.cmd" SHA256
```

Expected result. Three lines: `@echo off`; an `echo ... >> ...\heartbeat.log` line writing date, time and user; `ping -n 1 127.0.0.1 >nul`. The log has **two** lines (two runs), and you have a SHA-256 for the indicator list. **Explanation.** The script appends a heartbeat (a beacon-like pattern) and touches only loopback: no external communication in the code. Two log lines show two executions. **Verification.** The two timestamps differ by seconds to minutes. **Troubleshooting.** Never `type` unknown binaries; this is plain text. **Cleanup.** None. **Challenge.** Which IOCs would you share? **Solution.** File path and name, SHA-256, the log file path, the task name `CmdLabIR\SystemUpdateCheck`, the service name `CmdLabIRSvc` and display name `Windows Update Helper`, the registry key `HKCU\Software\CmdLabIR`.

Lab 139. Process and network evidence

Objective. Find which processes ran, from which parent, and judge the network question. **Difficulty.** [A]. **Prerequisites.** Lab 138. **Environment. VM**, elevated.

Commands.

```
wevtutil qe Security "/q:*[System[(EventID=4688)]]" /c:200 /rd:true /f:text | findstr /i
    .. /c:"updater.cmd" /c:"ping -n 1 127.0.0.1" /c:"Creator Process Name" /c:"Date:"
netstat -ano | findstr /i "ESTABLISHED"
ipconfig /displaydns | findstr /i "CmdLab"
type "C:\Users\Public\CmdLabIR\updater.cmd" | findstr /i "ping"
```

Expected result. 4688 events for `cmd.exe` with command lines containing `updater.cmd` (two: one launched from a console `cmd.exe`, one launched by the **Task Scheduler**, whose creator process is typically the `svchost.exe` hosting the Schedule service; verify on your build), plus child `PING.EXE` events with `127.0.0.1`. No unusual established connections now; nothing relevant in the DNS cache. **Explanation.** The event pair shows manual and scheduled execution. The only network action in the code is a ping to loopback.

Verification. Note both timestamps (T2 manual, T3 task). **Troubleshooting.** If 4688 command lines are blank, auditing/command-line capture was off when it ran (Lab 116). **Cleanup.** None. **Challenge.** State your conclusion on network activity and your confidence. **Solution.** *No evidence of external network activity; the script only pings loopback. Confidence: moderate, because connection auditing (5156) and DNS/proxy logs were not available to confirm negatives.*

Lab 140. Scheduled-task analysis

Objective. Fully characterize the task. **Difficulty.** [I]. **Prerequisites.** Lab 139. **Environment. VM**, elevated.

Commands.

```
schtasks /query /tn "CmdLabIR\SystemUpdateCheck" /v /fo list
schtasks /query /tn "CmdLabIR\SystemUpdateCheck" /xml > task_evidence.xml
findstr /i "Author Date Command UserId RunLevel Hidden TimeTrigger StartBoundary"
    .. task_evidence.xml
dir "%WINDIR%\System32\Tasks\CmdLabIR"
wevtutil qe Microsoft-Windows-TaskScheduler/Operational /c:20 /rd:true /f:text | findstr /i
    .. "Event ID: Task Name:"
```

Expected result. Task `\CmdLabIR\SystemUpdateCheck`; Author = the administrator account; one-time trigger (23:59); action runs `C:\Users\Public\CmdLabIR\updater.cmd`; principal = that user, not SYSTEM, not hidden; `Last Result 0x0`; log events show registration (106) then start/complete (200/201 style) for the manual `/run`. **Explanation.** Task name `SystemUpdateCheck` is designed to blend in; the action path under `C:\Users\Public` is the red flag. **Verification.** Registration time T1b close to T1. **Troubleshooting.** If no events appear, task history was not enabled until setup turned it on (it did) and rolled over. **Cleanup.** Keep `task_evidence.xml` as evidence. **Challenge.** List this task's suspicious characteristics. **Solution.** Non-vendor folder with a legitimate-sounding name, action in a user-writable public folder, script (interpreter) action, created shortly before first execution, hidden target file.

Lab 141. Service and registry analysis

Objective. Decide whether a service changed and what the registry shows. **Difficulty.** [A]. **Prerequisites.** Lab 140. **Environment. VM**, elevated.

Commands.

```
sc query CmdLabIRSvc
sc qc CmdLabIRSvc
reg query HKLM\SYSTEM\CurrentControlSet\Services\CmdLabIRSvc
dir "C:\Users\Public\CmdLabIR\Update Service"
wevtutil qe System "/q:*[System[(EventID=7045)]]" /c:3 /rd:true /f:text
reg query HKCU\Software\CmdLabIR
```

Expected result. Service `CmdLabIRSvc`, display name `Windows Update Helper`, state `STOPPED`, start type `DEMAND_START`, binary path `C:\Users\Public\CmdLabIR\Update Service\svc.exe` **unquoted with spaces**, account `LocalSystem`; the binary and the `Update Service` folder **do not exist**; event 7045 records the installation; the HKCU key has `Script` pointing at `updater.cmd`. **Explanation.** Findings: masquerading display name; unquoted path; user-writable location; high-privilege account; missing binary (never ran as a service; it would fail with error 1053 or file not found). It is an **installed but dormant** persistence-style entry.

Verification. Note the 7045 time T1c. **Troubleshooting.** `sc` must be `sc.exe` if you are in PowerShell. **Cleanup.** None. **Challenge.** Why is a service entry pointing to a missing file still a finding? **Solution.** If an attacker (or any user with write access to the folder or parent) later creates the binary, it would run as SYSTEM; the entry also shows intent and an entry-point gap (who was allowed to create services?).

Lab 142. Build the timeline

Objective. Merge sources into one ordered table and check it against your bracket. **Difficulty.** [A]. **Prerequisites.** Labs 137-141. **Environment. VM**.

Commands. Gather times (UTC where the source is UTC; convert consistently) and record in a table:

```
type lab136_start.txt
dir /a /t:c "C:\Users\Public\CmdLabIR"
dir /a /t:w "C:\Users\Public\CmdLabIR"
wevtutil qe Microsoft-Windows-TaskScheduler/Operational /c:10 /rd:true /f:text | findstr /i
    .. "Date: Event ID:"
wevtutil qe System "/q:*[System[(EventID=7045)]]" /c:1 /f:text | findstr /i "Date:"
wevtutil qe Security "/q:*[System[(EventID=4688)]]" /c:200 /f:text | findstr /i /c:"updater"
    .. /c:"Date:"
```

Expected result and complete solution. The order of events (all within a short window bracketed by Lab 136's timestamps):

Order	Event	Source
1	Evidence settings changed (audit policy, history enabled)	not logged as a distinct event in this lab
2	`CmdLabIR` folder and `updater.cmd` created (creation time T1), hidden attribute and stream added	file times
3	Scheduled task registered	TaskScheduler 106
4	Service `CmdLabIRSvc` installed	System 7045
5	Registry marker written	no timestamp in `reg query` (limitation)
6	Script run by hand from a console	Security 4688 (`cmd.exe ... updater.cmd`, parent `cmd.exe`)
7	Script run through the task	TaskScheduler 200/201 and 4688 with Task Scheduler parent
8	Last write of `heartbeat.log`	file time

Explanation. One coherent sequence shows installation first, then execution twice. **Verification.** Every time falls between the two stamps in `lab136_start.txt` (allowing for UTC vs local conversion). **Troubleshooting.** Event text times are local; XML is UTC; convert before merging. **Cleanup.** None. **Challenge.** Name two gaps in the timeline. **Solution.** The registry write time (not available from `reg query`; PowerShell or registry auditing could supply it) and the origin of the activity (nothing records how the files arrived; only the simulated `ZoneId` stream hints at it).

Lab 143. Volatile evidence collection

Objective. Run the Section 51.3 collector and validate the result. **Difficulty.** [A]. **Prerequisites.** Section 51.3. **Environment. VM**, elevated, output folder on a separate virtual disk if available (otherwise a local folder).

Commands.

```
collect_volatile.cmd "%USERPROFILE%\evidence\HOST01"
type "%USERPROFILE%\evidence\HOST01\collection_log.txt"
type "%USERPROFILE%\evidence\HOST01\MANIFEST_sha256.txt"
find /i "updater" "%USERPROFILE%\evidence\HOST01\08_process_details.csv"
find /i "CmdLabIR" "%USERPROFILE%\evidence\HOST01\11_tasks.csv"
find /i "Windows Update Helper" "%USERPROFILE%\evidence\HOST01\10_services.txt"
```

Expected result. A numbered set of evidence files, a log with step times, a SHA-256 manifest; the task appears in `11_tasks.csv` and the service in `10_services.txt`; `updater` is **not** in the process list (the script has finished) which shows why persistent artifacts matter. **Explanation.** The collector captured the system's current state in volatility order; the earlier labs reconstructed the past from persistent traces. **Verification.** Re-hash one file with `certutil -hashfile` and compare with the manifest; also confirm every file is non-empty or has an explained error. **Troubleshooting.** `quser` can fail on some editions; that error text is itself recorded. **Cleanup.** Keep the folder until Lab 145. **Challenge.** List what the collector changed on the system. **Solution.** New processes and Prefetch entries for the tools, files in the output folder, entries in the Security log (4688 for each tool), an audit trail of your own actions: document them in the report.

Lab 144. Preserve, contain, eradicate, verify

Objective. Remove the simulated persistence in the correct order with evidence intact. **Difficulty.** [A]. **Prerequisites.** Labs 137-143. **Environment. VM**, elevated.

Commands.

```
md "%USERPROFILE%\evidence\HOST01\artifacts"
rem 1. PRESERVE
copy "task_evidence.xml" "%USERPROFILE%\evidence\HOST01\artifacts\"
sc qc CmdLabIRSvc > "%USERPROFILE%\evidence\HOST01\artifacts\service_qc.txt"
reg export HKCU\Software\CmdLabIR "%USERPROFILE%\evidence\HOST01\artifacts\hkcu_cmdlabir.reg"
    .. /y
reg export HKLM\SYSTEM\CurrentControlSet\Services\CmdLabIRSvc
    .. "%USERPROFILE%\evidence\HOST01\artifacts\svc_key.reg" /y
attrib -h "C:\Users\Public\CmdLabIR\updater.cmd"
robocopy "C:\Users\Public\CmdLabIR" "%USERPROFILE%\evidence\HOST01\artifacts\files" /e
    .. /copy:DAT /r:1 /w:1 /np /njh /njs
certutil -hashfile "%USERPROFILE%\evidence\HOST01\artifacts\files\updater.cmd" SHA256
rem 2. CONTAIN (reversible)
schtasks /change /tn "CmdLabIR\SystemUpdateCheck" /disable
sc config CmdLabIRSvc start= disabled
rem 3. ERADICATE
schtasks /delete /tn "CmdLabIR\SystemUpdateCheck" /f
sc delete CmdLabIRSvc
reg delete HKCU\Software\CmdLabIR /f
rd /s /q "C:\Users\Public\CmdLabIR"
rem 4. VERIFY
schtasks /query /tn "CmdLabIR\SystemUpdateCheck"
sc query CmdLabIRSvc
reg query HKCU\Software\CmdLabIR
dir C:\Users\Public\CmdLabIR
schtasks /query /fo csv | findstr /i "CmdLabIR"
```

Expected result. Every verification command reports not found (`ERROR: The system cannot find the file specified.`, error 1060, key not found, file not found) and the last `findstr` prints nothing; the artifacts folder holds the preserved copies. **Explanation.** Preserve, contain, eradicate, verify. Disabling comes before deleting; copying happens before any deletion. **Verification.** `certutil -hashfile` of the preserved script matches the hash from Lab 138. **Troubleshooting.** If `sc delete` reports "marked for deletion", close tools that hold the service handle (services.msc, Process Explorer) and retry or reboot. **Cleanup.** Run `irsim_teardown.cmd` to restore auditing/history settings and clear any remains. **Challenge.** What is still unfinished after this lab? **Solution.** Root cause and entry point (how the files were placed), checking other hosts, resetting credentials if exposure is suspected, and monitoring for recurrence.

Lab 145. Final report and lessons learned

Objective. Answer the scenario's eleven questions and write the report. **Difficulty.** [A]. **Prerequisites.** Labs 136-144. **Environment.** Notepad or a text file.

Commands. Fill in `incident_report.txt` using the Section 50.8 template; then run `irsim_teardown.cmd` and verify removal.

Expected result and complete solution (answer key).

Question	Answer
Who executed it?	The administrator account that ran the setup: manual run from a console `cmd.exe` (4688 Subject user) and a task run under that user's principal (task `Author`/`UserId`)
When?	Within the bracket recorded in `lab136_start.txt`; first creation at T1, executions at T2 (manual) and T3 (task), last log write at the second run
What commands ran?	`cmd.exe /c ""C:\Users\Public\CmdLabIR\updater.cmd""` and its content: append heartbeat line to `heartbeat.log`; `ping -n 1 127.0.0.1`

Question	Answer
Which processes were created?	`cmd.exe` (twice, different parents) and `PING.EXE` (twice); no unknown binaries
What files changed?	Created `C:\Users\Public\CmdLabIR\updater.cmd` (hidden, with `origin` stream), `heartbeat.log`; task XML under `System32\Tasks\CmdLabIR`
Scheduled task?	Yes: `\CmdLabIR\SystemUpdateCheck`, one-time 23:59 trigger, action = the script, runs as the creator
Service changed?	Yes: `CmdLabIRSvc` ("Windows Update Helper"), demand start, LocalSystem, unquoted path under Public, binary missing, never started
Network activity?	No evidence of external activity; only a loopback ping in the script. Confidence moderate (no connection auditing)
Evidence remaining	File times and owner; 4688 (with command lines because setup enabled capture); TaskScheduler events 106/200/201; System 7045; task XML; registry key; collection folder with hashes
Evidence gaps	No registry write time; no connection auditing; no record of how the files arrived
Remediation	Lab 144 order: preserve, disable, delete task/service/registry/files, verify; then close the entry point and monitor
Verification of cleanup	All queries return not found; baseline compare; monitor for 7045/4698/106 events and the indicators (hash, path, names)
Root cause (simulation)	Created by the exercise's setup; in a real case: how did a script get into Public and who could register tasks and services?
Recommendations	Alert on new services and tasks; block script execution from `Users\Public`; tighten ACLs; enable process-creation and connection auditing; forward logs; standard accounts for daily work

Explanation. The report answers every question and states confidence and gaps honestly. **Verification.** `schtasks /query /tn "CmdLabIR\SystemUpdateCheck"`, `sc query CmdLabIRSvc`, `reg query HKCU\Software\CmdLabIR` all report not found after teardown. **Troubleshooting.** If the teardown leaves the empty task folder, it is harmless; check with `schtasks /query /fo list | find /i "CmdLabIR"`. **Cleanup.** Teardown script; revert the VM snapshot if in doubt; keep `incident_report.txt` as a model. **Challenge.** Write three detection rules that would have caught this sooner. **Solution.** (1) Alert on event 7045 whose `ImagePath` is in a user-writable path or unquoted with spaces; (2) alert on 4698/106 where the action path is under `Users\Public`, `AppData` or `Temp`; (3) alert on `cmd.exe` launched by Task Scheduler running a `.cmd` from a non-admin-writable-denied path, or on any script executed from `C:\Users\Public`.

Batch Scripting Labs (continued)

Labs 71-80 complete the batch-scripting lab set that began in Part V.

Chapter 53. Batch Labs 71-80

SAFETY. Scratch folder only. Scripts here use delayed expansion for data, so avoid input containing `!` or unmatched quotes (Section 47.5). They are teaching tools, not hardened production code.

Lab 71. String functions: length, trim, upper-case

Objective. Build reusable string subroutines. **Difficulty.** [I]. **Prerequisites.** Section 16.1. **Environment.** Scratch folder.

Commands. Save as `strlib.cmd`:

```
@echo off
setlocal EnableExtensions
call :strlen "hello world" L
echo length=%L%
call :trim "   padded text   " T
echo [%T%]
call :upper "Mixed Case 123" U
echo %U%
exit /b 0

:strlen
setlocal EnableDelayedExpansion
set "S=%~1"
set /a N=0
:strlen_loop
if defined S (set "S=!S:~1!" & set /a N+=1 & goto :strlen_loop)
endlocal & set "%~2=%N%"
exit /b 0

:trim
setlocal EnableDelayedExpansion
set "S=%~1"
:trim_l
if defined S if "!S:~0,1!"==" " (set "S=!S:~1!" & goto :trim_l)
:trim_r
if defined S if "!S:~-1!"==" " (set "S=!S:~0,-1!" & goto :trim_r)
endlocal & set "%~2=%S%"
exit /b 0

:upper
setlocal EnableDelayedExpansion
set "S=%~1"
for %%c in (A B C D E F G H I J K L M N O P Q R S T U V W X Y Z) do set "S=!S:%%c=%%c!"
endlocal & set "%~2=%S%"
exit /b 0
```

Expected result. `length=11`, `[padded text]`, `MIXED CASE 123`. **Explanation.** `endlocal & set` returns a value across the scope boundary because `%N%`/`%S%` are expanded when that single line is parsed, before `endlocal` runs. Substitution `!S:A=A!` is case-insensitive, which makes the upper-case trick work. **Verification.** Test an empty string and a string of only spaces. **Troubleshooting.** Values containing `"` or `!` break these routines; document that limit. **Cleanup.** `del strlib.cmd`. **Challenge.** Write `:lower`. **Solution.** Same loop with lower-case letters: `set "S=!S:%%c=%%c!"` over `a b c ...`.

Lab 72. Locale-proof timestamps

Objective. Produce `yyyyMMdd-HHmmss` regardless of regional settings. **Difficulty.** [I]. **Prerequisites.** Section 22.7. **Environment.** Scratch folder.

Commands.

```
for /f %t in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd-HHmmss"') do @set
    .. "STAMP=%t"
echo %STAMP%
echo %DATE% %TIME%
> report_%STAMP%.txt echo report created %STAMP%
dir /b report_*.txt
```

Expected result. A stamp like `20261002-141530`, a regional date on the next line, and a file named with the stamp. **Explanation.** `%DATE%` format varies by region, so file names built from it break on other machines. **Verification.** Run twice a second apart: names sort chronologically. **Troubleshooting.** The `@set` is needed so the loop does not echo; in a script use `%%t`. **Cleanup.** `del report_*.txt & set STAMP=`. **Challenge.** Compute whether a file is older than 7 days without PowerShell. **Solution.** `forfiles /p . /m file.txt /d -7 >nul 2>&1 && echo older than 7 days`.

Lab 73. CSV reporter that survives empty fields

Objective. Fix the FOR /F collapsing-delimiter problem. **Difficulty.** [A]. **Prerequisites.** Section 15.4. **Environment.** Scratch folder.

Commands. Create data and save the script:

```
> people.csv echo name,city,age
>> people.csv echo Alice,Paris,30
>> people.csv echo Bob,,25
>> people.csv echo Carol,Rome,
```

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
echo --- naive
for /f "skip=1 tokens=1-3 delims=," %%a in (people.csv) do echo name=%%a city=%%b age=%%c
echo --- fixed
for /f "usebackq skip=1 delims=" %%L in ("people.csv") do (
    set "L=%%L"
    set "L=!L:,,=, ,!"
    set "L=!L:,,=, ,!"
    for /f "tokens=1-3 delims=," %%a in ("!L!") do echo name=%%a city=%%b age=%%c
)
```

Expected result. The naive loop misaligns Bob (`city=25` and a literal `age=%c`, because consecutive commas collapse) and shows a literal `age=%c` for Carol (missing third token); the fixed loop keeps Bob's fields in the right columns (`city=` followed by a space). Carol still shows `age=%c` until you apply the trailing-comma fix from Troubleshooting. **Explanation.** `for /f` treats runs of delimiters as one. Padding empty fields with a space restores alignment. **Verification.** Compare the two outputs line by line. **Troubleshooting.** A trailing comma (Carol) gives a missing third token; handle it by appending a space: `set "L=!L! "` if the line ends with a comma. **Cleanup.** `del people.csv`. **Challenge.** Handle quoted fields containing commas. **Solution.** Not

reliably possible in CMD; use PowerShell `Import-Csv`.

Lab 74. Menu with CHOICE and GOTO dispatch

Objective. Write an interactive menu with correct errorlevel ordering. **Difficulty.** [B]. **Prerequisites.** `choice` card. **Environment.** Scratch folder.

Commands. Save as `menu.cmd`:

```
@echo off
setlocal EnableExtensions
:menu
cls
echo ===== Lab menu =====
echo   1) Show date and time
echo   2) Show host name
echo   3) Quit
choice /c 123 /n /m "Select 1-3: "
if errorlevel 3 goto :quit
if errorlevel 2 goto :host
if errorlevel 1 goto :dt

:dt
echo %DATE% %TIME%
pause
goto :menu

:host
hostname
pause
goto :menu

:quit
echo bye
exit /b 0
```

Expected result. Pressing 1, 2 or 3 performs the action; other keys are ignored. **Explanation.** `if errorlevel N` tests "N or higher", so test the highest value first. **Verification.** Press 3 first, then 1, then 2. **Troubleshooting.** `choice` must be used with `/n` to hide the key list if you print your own prompt. **Cleanup.** `del menu.cmd`. **Challenge.** Add a 10-second timeout that defaults to Quit. **Solution.** `choice /c 123 /n /t 10 /d 3 /m "Select 1-3: "`.

Lab 75. Option parser for `/name:value` arguments

Objective. Parse mixed options and positional arguments. **Difficulty.** [A]. **Prerequisites.** Section 21.3. **Environment.** Scratch folder.

Commands. Save as `opts.cmd`:

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
:loop
if "%~1"=="" goto :done
set "A=%~1"
if "!A:~0,1!"=="/" (
    set "KV=!A:~1!"
    if "!KV::=!"=="!KV!" (
        set "OPT_!KV!=1"
    ) else (
        for /f "tokens=1,* delims=:" %%k in ("!KV!") do set "OPT_%%k=%%l"
    )
) else (
    echo positional: %~1
)
shift
goto :loop
:done
echo --- options found
set OPT_
exit /b 0
```

Run `opts /name:Alice /count:3 file1.txt "second file.txt" /flag`.

Expected result. Two positional lines and a list `OPT_count=3`, `OPT_flag=1` (a switch with no value is stored as 1), `OPT_name=Alice`. **Explanation.** The `OPT_` prefix prevents an option from overwriting critical variables. **Verification.** Try `/path:C:\temp`: the first colon splits, the rest (`C:\temp`) is preserved. **Troubleshooting.** Arguments containing `!` are damaged by delayed expansion. **Cleanup.** `del opts.cmd`. **Challenge.** Reject unknown options. **Solution.** Keep an allowlist string and test `echo !ALLOWED! | find /i "%%k"` (with exact-token matching) before storing.

Lab 76. New-process detector

Objective. Report processes that appeared since a baseline. **Difficulty.** [I]. **Prerequisites.** Lab 83. **Environment.** Scratch folder.

Commands. Save as `newproc.cmd`:

```
@echo off
setlocal EnableExtensions
cd /d "%~dp0"
set "BASE=proc_base.txt"
set "NOW=proc_now.txt"
(for /f "tokens=1 delims=," %%n in ('tasklist /fo csv /nh') do @echo %%~n) | sort > "%NOW%"
if /i "%~1"=="/base" (copy /y "%NOW%" "%BASE%" >nul & echo baseline saved & exit /b 0)
if not exist "%BASE%" (echo run with /base first & exit /b 2)
echo --- new process names since baseline
findstr /l /x /v /g:%BASE% %NOW%
```

Run `newproc /base`, start `notepad`, run `newproc`.

Expected result. `notepad.exe` appears under the new-processes heading (plus any other process that started). **Explanation.** A name-level diff is noisy but cheap; real tools also compare path and parent. **Verification.** Close Notepad and run again: no new entries. **Troubleshooting.** Short-lived processes between runs are missed. **Cleanup.** `del newproc.cmd proc_base.txt proc_now.txt`. **Challenge.** Include executable paths. **Solution.** Use the CIM process list with `ExecutablePath` and diff on that column.

Lab 77. Disk-space check with thresholds and exit codes

Objective. Return meaningful exit codes for monitoring. **Difficulty.** [I]. **Prerequisites.** Section 16.2. **Environment.** Scratch folder.

Commands. Save as `diskcheck.cmd`:

```
@echo off
setlocal EnableExtensions
set "DRIVE=%~1"
if not defined DRIVE set "DRIVE=C"
set "WARN=%~2"
if not defined WARN set "WARN=20"
set "GB="
for /f %%g in ('powershell -NoProfile -Command "[int]((Get-PSDrive %DRIVE%).Free/1GB)"') do
    .. set "GB=%%g"
if not defined GB (echo ERROR: cannot read drive %DRIVE% & exit /b 3)
echo Free space on %DRIVE%: %GB% GB (warn below %WARN% GB)
if %GB% lss %WARN% (echo WARNING: low disk space & exit /b 1)
echo OK
exit /b 0
```

Run `diskcheck`, `diskcheck C 100000`, `diskcheck Z`.

Expected result. OK with exit 0; WARNING with exit 1 when the threshold is huge; ERROR with exit 3 for a missing drive. **Explanation.** PowerShell supplies the large number because `set /a` is limited to 32 bits (about 2 billion). **Verification.** `echo %ERRORLEVEL%` after each call, on its own line. **Troubleshooting.** The `%DRIVE%` value goes into a PowerShell command, so validate it: accept only a single letter. **Cleanup.** `del diskcheck.cmd`. **Challenge.** Add the drive-letter validation. **Solution.** `echo %DRIVE%| findstr /r /x "[A-Za-z]" >nul || (echo bad drive & exit /b 2)` before use (acceptable here because the value is a single token from the caller who already has the same rights; in a trust boundary use the validation pattern from Section 47.3).

Lab 78. Safe purge with dry-run default

Objective. Delete old files only after a preview. **Difficulty.** [I]. **Prerequisites.** `forfiles` card. **Environment.** Scratch folder.

Commands. Create test files with old dates, then save `purge.cmd`:

```
md purge_test
for %i in (1 2 3) do @echo x> purge_test\old%i.log
for %i in (1 2) do @echo x> purge_test\new%i.log
powershell -NoProfile -Command "Get-ChildItem purge_test\old*.log | ForEach-Object {
    .. $_.LastWriteTime = (Get-Date).AddDays(-40) }"
```

```
@echo off
setlocal EnableExtensions
set "DIR=%~1"
set "DAYS=%~2"
if not exist "%DIR%\" (echo Usage: %~nx0 folder days [/go] & exit /b 2)
echo %DAYS%| findstr /r /x "[0-9][0-9]*" >nul || (echo days must be a number & exit /b 2)
if /i "%~3"=="/go" (
    echo DELETING *.log older than %DAYS% days in %DIR%
    forfiles /p "%DIR%" /m *.log /d -%DAYS% /c "cmd /c echo deleting @path & del @path"
) else (
    echo DRY RUN: would delete the following (add /go to apply)
    forfiles /p "%DIR%" /m *.log /d -%DAYS% /c "cmd /c echo @path"
)
exit /b 0
```

Run `purge purge_test 30`, then `purge purge_test 30 /go`, then `dir /b purge_test`.

Expected result. The dry run lists `old1`-`old3`; the real run deletes them; `new1` and `new2` remain. **Explanation.** Dry-run default plus input validation plus a preview of the exact file set. **Verification.** Final `dir /b`. **Troubleshooting.** If nothing is listed, the date change may have failed; check `dir /t:w purge_test`. **Cleanup.** `rd /s /q purge_test & del purge.cmd`. **Challenge.** Log each deletion with a timestamp. **Solution.** Add `>> purge.log echo %DATE% %TIME% deleted` inside the `/c` command, or wrap with the `:log` function.

Lab 79. Retry wrapper

Objective. Run any command with retries, delay and a clear exit code. **Difficulty.** [A]. **Prerequisites.** Section 16.6. **Environment.** Scratch folder.

Commands. Save as `retry.cmd`:

```
@echo off
setlocal EnableExtensions
set "TRIES=%~1"
set "DELAY=%~2"
set "CMDLINE=%~3"
if "%CMDLINE%"=="" (echo Usage: %~nx0 tries delay_seconds "command" & exit /b 2)
echo %TRIES%| findstr /r /x "[0-9][0-9]*" >nul || (echo tries must be a number & exit /b 2)
echo %DELAY%| findstr /r /x "[0-9][0-9]*" >nul || (echo delay must be a number & exit /b 2)
set /a N=0
:try
set /a N+=1
echo attempt %N% of %TRIES%
cmd /c %CMDLINE%
if not errorlevel 1 (echo succeeded & exit /b 0)
if %N% geq %TRIES% (echo giving up after %N% attempts & exit /b 1)
set /a D=%DELAY%+1
ping -n %D% 127.0.0.1 >nul
goto :try
```

Run `retry 3 1 "ping -n 1 127.0.0.1"` and `retry 3 1 "ping -n 1 no-such-host.invalid"`.

Expected result. First: one attempt, `succeeded`, exit 0. Second: three attempts about a second apart, `giving up after 3 attempts`, exit 1. **Explanation.** `ping -n D` waits D-1 seconds, so the script adds 1. **Verification.** `echo %ERRORLEVEL%` after each. **Troubleshooting.** The command text is passed to `cmd /c`, so the caller can run anything they could type themselves: that is acceptable for personal tooling, but **never expose such a wrapper to less-trusted input**. **Cleanup.** `del retry.cmd`. **Challenge.** Add a per-attempt timeout. **Solution.**

Not possible in pure CMD without helper tools; use `start /wait` with a watchdog loop or PowerShell `Wait-Process -Timeout`.

Lab 80. Configuration-driven backup (mini project)

Objective. Combine config parsing, validation, robocopy exit handling and logging. **Difficulty.** [A]. **Prerequisites.** Labs 38, 63, 67. **Environment.** Scratch folder.

Commands. Create `backup.ini` and save the script as `backup.cmd`:

```
md bk_src & echo one> bk_src\a.txt & echo two> bk_src\b.txt
> backup.ini echo # backup settings
>> backup.ini echo src=bk_src
>> backup.ini echo dst=bk_dst
>> backup.ini echo retries=2
```

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
set "CFG=%~dp0backup.ini"
set "LOG=%~dp0backup.log"
if not exist "%CFG%" (echo missing config & exit /b 3)
for /f "usebackq eol=# tokens=1,* delims==" %%k in ("%CFG%") do set "CFG_%%k=%%l"
for %%v in (src dst retries) do if not defined CFG_%%v (echo config missing %%v & exit /b 3)
echo !CFG_retries!| findstr /r /x "[0-9][0-9]*" >nul || (echo retries must be a number & exit
    .. /b 2)
if not exist "!CFG_src!\" (echo source not found: !CFG_src! & exit /b 3)
if /i "!CFG_src!"=="!CFG_dst!" (echo source and destination are identical & exit /b 2)
>> "%LOG%" echo %DATE% %TIME% START src=!CFG_src! dst=!CFG_dst!
robocopy "!CFG_src!" "!CFG_dst!" /e /r:!CFG_retries! /w:1 /np /njh /njs >> "%LOG%" 2>&1
set "RC=%ERRORLEVEL%"
>> "%LOG%" echo %DATE% %TIME% END robocopy code !RC!
if !RC! geq 8 (echo BACKUP FAILED, see %LOG% & exit /b 1)
fc "!CFG_src!\a.txt" "!CFG_dst!\a.txt" >nul && echo backup verified (a.txt identical)
exit /b 0
```

Run it twice and `type backup.log`.

Expected result. First run copies (robocopy code 1) and verifies; second run reports code 0. The log shows START/END pairs. **Explanation.** Every external input (config values) is validated before use; a swapped or identical source and destination is rejected; the robocopy exit code is interpreted correctly; the result is verified. **Verification.** `dir /b bk_dst`. **Troubleshooting.** Values with spaces in `src=` are handled by the quotes around `!CFG_src!`, but avoid `!`. **Cleanup.** `rd /s /q bk_src bk_dst & del backup.ini backup.log backup.cmd`. **Challenge.** Add `exclude=` patterns from the config. **Solution.** Validate the value with an allowlist, then pass it as `/xf "!CFG_exclude!"`.

Back Matter (Volume 3)

Glossary additions, command index, references, items to verify, lab inventory and the plan for Volume 4.

Chapter 54. Glossary Additions

Term	Definition
Allowlist	A validation approach that accepts only known-good input; stronger than blocking known-bad input
Application control	Policy (WDAC or AppLocker) that decides which executables, scripts and DLLs may run
ASR rules	Attack Surface Reduction: Defender rules that block risky behaviors such as Office launching child processes
Argument injection	Input that changes a tool's behavior by being interpreted as an option or switch
Chain of custody	Documented record of who held evidence, when, and what was done to it
Command injection	Input that is interpreted as shell syntax rather than data
Containment	Limiting the spread or effect of an incident without destroying evidence
Dwell time	How long unauthorized activity went undetected
Eradication	Removing the cause and persistence mechanisms of an incident
IOC	Indicator of compromise: an observable (hash, path, name, address) associated with malicious activity
LOLBin / LOLBAS	Legitimate built-in binary used in an unintended way / the project cataloging them
Masquerading	Making something look like a legitimate file or process (name or path)
Mark of the Web	`Zone.Identifier` stream recording that a file came from the Internet
Order of volatility	Collect the most short-lived evidence first
Path interception	Causing a different program than intended to run by controlling a searched location
Persistence	A mechanism that makes code run again after logoff or reboot
Second-order injection	Data stored safely but later interpreted unsafely (for example in a generated script)
Sigma	A generic rule format for describing detections in logs
Sysmon	Sysinternals system monitor that writes detailed process, network and registry events
Telemetry	Recorded observations of system activity used for detection and investigation
Trust boundary	A line across which data or control passes between different privilege or trust levels
WDAC	Windows Defender Application Control: kernel-enforced code-integrity policy

Chapter 55. Command Index (Volume 3)

Command	Where
`bitsadmin`	Chapter 46
`certutil`	Chapter 46
`cscript`, `wscript`	Chapter 46
`forfiles`	Chapter 46
`installutil`	Chapter 46
`mshta`	Chapter 46
`msiexec`	Chapter 46
`regsvr32`	Chapter 46
`rundll32`	Chapter 46
`wmic` (deprecated)	Chapter 46
`auditpol` (telemetry setup)	Chapter 44
Collection tools (`netstat`, `arp`, `ipconfig /displaydns`, `tasklist`, `quser`, `net session`, `systeminfo`, `wevtutil epl`)	Chapter 51

Still to be carded in Volume 4: `driverquery`, `query`, `quser`, `qwinsta`, `openfiles`, `shutdown`, `diskpart`, `mountvol`, `format`, `label`, `convert`, `defrag`, `expand`, `makecab`, `replace`, `telnet`, `ftp`, `mode`, `break`, `verify`, `subst`.

Chapter 56. References and Verification Notes

Primary and authoritative sources (verify against current versions):

- Microsoft Learn: Windows commands; Windows security (access tokens, ACLs, UAC); Windows Defender Application Control and AppLocker; Attack surface reduction rules; Microsoft Defender Antivirus; Windows Security Auditing event reference (4624, 4625, 4688, 4697, 4698, 4720, 4732, 1102); *Advanced security audit policy*; *Command line process auditing* (policy to include command lines in process-creation events).
- Microsoft Sysinternals: Sysmon (event IDs 1, 3, 11, 12, 13, 22), Process Explorer, Process Monitor, Autoruns, AccessChk, Streams, Sigcheck.
- MITRE ATT&CK (techniques used as vocabulary): T1059.003, T1036, T1574, T1053.005, T1543.003, T1547.001, T1070.001, T1564.004, T1218 (System Binary Proxy Execution), T1105 (Ingress Tool Transfer), T1197 (BITS Jobs), T1047 (WMI), T1552 (Unsecured Credentials).
- LOLBAS project (lolbas-project.github.io): catalog of dual-use Windows binaries (used for detection engineering; this book does not reproduce abuse command lines).
- NIST SP 800-61 *Computer Security Incident Handling Guide*; NIST SP 800-86 *Guide to Integrating Forensic Techniques into Incident Response*.
- EICAR (eicar.org) for the standard anti-malware test file.

Items to verify on your own machine. I have not executed any command or lab on Windows.

- The exact 4688 field names and text layout in your build (`Creator Process Name`, `Process Command Line`), and whether Task Scheduler launches report `svchost.exe` as the creator.
- That `echo ^^winword...` writes a literal `^` (Lab 125) and that `findstr /g` file patterns behave as described.
- `cmd` behavior for unquoted paths with spaces (Labs 47 and 122) on your build.
- `Get-AuthenticodeSignature` status for copied OS binaries (Lab 128).
- WMI-Activity, BITS-Client and Defender event IDs mentioned in cards (check the event provider manifests on your system).
- The simulator (Lab 136): that `schtasks /run` completes before the script reads the log, and that teardown fully restores audit and history settings.
- WMIC, VBScript and `mshta` availability on current Windows 11 builds.

Safety recap. The injection labs use a marker (`INJECTED-MARKER`), never a real payload. The LOLBin cards describe detection and mitigation and omit abuse syntax. The incident simulation creates only harmless, named objects and has a teardown. Log clearing (Lab 132) is VM-only and exports first.

Chapter 57. Lab Inventory After Volume 3

Range	Status
1-25	Volume 1
26-40	Volume 2
41-65	Volume 1
66-70	Volume 2
71-80	**Volume 3**
81-112	Volume 2
113-115	Not yet written (networking; Volume 4)
116-135	**Volume 3** (security)
136-145	**Volume 3** (incident response)
146-150	**Volume 3** (injection and script review)
151-165	Volume 2 (logs, Defender, recovery, performance)

Total written so far: 162 labs.

Chapter 58. What Comes Next

Volume 4 (planned): labs 113-115; the 17 major projects with complete source (system information collector, hardware and software inventory, network diagnostic suite, health checker, backup tool, service, task, user/group, NTFS permission and firewall auditors, event-log collector, Defender checker, baseline/diff tool, batch test framework, incident-response evidence collector, and the integrated assessment toolkit); the final capstone **Windows CLI Audit and Security Assessment Toolkit** with the specified report layout; the troubleshooting encyclopedia; expert reference tables (operators, environment variables, extensions, exit codes, common errors); the CMD versus PowerShell comparison; the remaining command cards; and the combined glossary, master command index and bibliography.